

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждения образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

**ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
ПРОЕКТА**

Тема «Проектирование и расчет объемного гидропривода с использованием программного комплекса»

Исполнитель

Студентка 3 курса 1 группы _____ Водчиц Анастасия Витальевна
(Ф.И.О.)

Руководитель работы _____ Блинова Елена Александровна
(Ф.И.О.)

Минск 2025

Содержание

Введение	5
1 Теоретические основы расчета объемного гидропривода	6
1.1 Основные допущения при расчете гидропривода.....	6
1.2 Классификация и назначение элементов объемного гидропривода	6
1.2.1 Насосы	7
1.2.2 Гидродвигатели.....	7
1.2.3 Гидроаппаратура.....	7
1.2.4 Вспомогательные устройства.....	8
1.3 Определение мощности гидропривода и выбор насоса	9
1.3.2 Определение полезной мощности насоса с учетом коэффициентов запаса.....	10
1.3.3 Выбор насоса по номинальному давлению и рабочему объему	10
1.4 Расчет и выбор гидроцилиндров.....	11
1.4.1 Определение диаметров поршня и штока.....	11
1.4.2 Расчет скоростей и расходов	13
1.5 Расчет и выбор гидромоторов	14
1.6 Определение внутреннего диаметра гидролиний и скоростей движения жидкости.....	15
1.7 Расчет потерь давления в гидролиниях и гидроаппаратуре	16
1.7.1 Потери давления на трение в трубопроводах.....	16
1.7.2 Потери давления в местных сопротивлениях.....	17
1.7.3 Потери давления в гидроаппаратуре	17
1.8 Определение КПД гидропривода	18
1.9 Тепловой расчет гидропривода.....	18
1.10 Выбор рабочей жидкости и ее основные характеристики	19
1.11 Выводы по разделу	20
2 Разработка программного комплекса для проектирования и расчета гидравлических схем	21
2.1 Общее описание архитектуры программного комплекса	21
2.2.1 Обзор используемых технологий (React, DND).....	21
2.2.2 Компонент каталога модулей.....	22
2.2.3 Основное рабочее поле	24
2.2.4 Панель свойств выбранного модуля («PropertiesScreen.js»).....	40
2.2.5 Модальное окно загрузки схем («LoadSchemeModal.js»)	46
2.2.6 Панель отображения результатов	49
2.3 Реализация серверной логики (Backend).....	55

2.3.1 Обзор используемых технологий (Node.js, Express).....	55
2.3.2 API для взаимодействия с клиентской частью.....	55
2.3.3 Алгоритм обработки гидравлической схемы	57
2.3.4 Реализация гидравлических расчетов на сервере	62
2.3.5 Реализация теплового расчета на сервере.....	66
2.4 Работа с базой данных.....	67
2.4.1 Схема данных для сохранения гидравлических схем	67
2.4.2 Операции CRUD для схем	69
2.5 Выводы по разделу	70
3 Проектный расчет объемного гидропривода.....	72
3.1 Общие исходные данные для примеров расчета.....	72
3.2 Пример 1: Расчет гидронавесной системы (ГНС).....	72
3.2.1 Описание схемы и исходных данных.....	72
3.2.2 Результаты расчета и их анализ	73
3.3 Пример 2: Расчет гидросистемы рулевого управления (ГРУ).....	74
3.3.1 Описание схемы и исходных данных.....	74
3.3.2 Результаты расчета и их анализ	74
3.4 Пример 3: Расчет комбинированной гидросистемы	75
3.4.1 Описание схемы и исходных данных.....	75
3.4.2 Результаты расчета и их анализ	75
3.5 Выводы по разделу	77
Заключение	78
Список используемых источников	79

Введение

Объемные гидроприводы (ОГП) находят все более широкое применение в современной технике благодаря своим уникальным эксплуатационным характеристикам. Высокая плотность передаваемой мощности, возможность бесступенчатого регулирования скорости выходного звена, надежная защита от перегрузок, компактность и сравнительная простота реверсирования делают их незаменимыми во многих отраслях промышленности, особенно в строительных, дорожных и подъемно-транспортных машинах. Актуальность применения ОГП обусловлена постоянным стремлением к повышению производительности, точности и надежности машин, а также к улучшению условий труда операторов. Широкое применение гидравлического привода является одним из наиболее эффективных путей повышения производительности строительных, дорожных и подъемно-транспортных машин, и в настоящее время около 80% из них оснащено гидроприводом.

Целью настоящего курсового проекта является углубление теоретических знаний и приобретение практических навыков в области проектирования и расчета объемных гидроприводов. В рамках проекта решаются следующие задачи:

- Изучение теоретических основ функционирования и расчета элементов объемного гидропривода.
- Разработка и анализ принципиальных гидравлических схем для различных типов исполнительных механизмов (с поступательным и вращательным движением).
- Выбор типовых элементов гидропривода (насосов, гидродвигателей, гидроаппаратуры, гидробаков, трубопроводов) на основе расчетных параметров.
- Определение основных эксплуатационных характеристик проектируемого гидропривода, включая потери давления, КПД и тепловой баланс.
- Разработка программного комплекса, автоматизирующего процесс проектирования, расчета и анализа гидравлических систем.

Ключевой проблемой, решаемой в данном проекте, является комплексный расчет параметров гидросистемы, начиная от определения требуемых усилий и скоростей на исполнительных органах и заканчивая выбором конкретных компонентов и оценкой общей эффективности привода. Проектный расчет включает определение мощности гидропривода, выбор насоса и гидродвигателей, расчет диаметров гидролиний и скоростей движения жидкости, подбор гидроаппаратуры и кондиционеров рабочей жидкости, а также расчет потерь давления и КПД гидропривода.

Для решения поставленных задач был разработан программный комплекс, состоящий из клиентской части (пользовательский интерфейс для построения схем и ввода параметров) и серверной части (выполнение расчетов и управление данными). Клиентская часть позволяет пользователю интерактивно создавать гидравлические схемы путем перетаскивания модулей из каталога на рабочее поле и их соединения. Серверная часть принимает данные о схеме и параметрах ее элементов, производит все необходимые гидравлические и тепловые расчеты, и возвращает результаты пользователю.

1 Теоретические основы расчета объемного гидропривода

Проектирование и расчет объемного гидропривода (ОГП) основывается на фундаментальных законах гидравлики и механики. Для упрощения инженерных расчетов и получения практически применимых результатов принимается ряд допущений, а также используется устоявшаяся классификация элементов гидропривода.

1.1 Основные допущения при расчете гидропривода

При расчете объемного гидропривода принимается ряд допущений, основными из которых являются:

- Рабочая жидкость считается несжимаемой. Это допущение справедливо для большинства инженерных расчетов, так как сжимаемость минеральных масел при рабочих давлениях незначительна и ее учет существенно усложняет расчеты без значительного повышения точности для большинства практических задач.

- Температура жидкости и ее основные физические свойства (плотность, вязкость и др.) принимаются постоянными. В реальных условиях температура жидкости может изменяться, что влияет на ее вязкость и, следовательно, на гидравлические потери. Однако для предварительных и основных расчетов часто принимают усредненные значения этих параметров, соответствующие установившемуся тепловому режиму.

- Режим работы гидропривода – установившийся. Это означает, что параметры потока (давление, скорость, расход) в каждой точке системы не изменяются во времени. Расчеты переходных процессов (пуск, остановка, изменение нагрузки) требуют более сложных методов анализа.

- Коэффициенты гидравлических сопротивлений постоянны. Хотя эти коэффициенты могут зависеть от числа Рейнольдса и, следовательно, от скорости потока и вязкости жидкости, для установившегося режима их часто принимают постоянными для заданного элемента или участка трубопровода.

- Разрыва потока жидкости при работе гидропривода не происходит. Это допущение означает непрерывность потока во всех элементах гидросистемы.

- Подача насоса, питающего гидросистему, считается постоянной (для нерегулируемых насосов при постоянной частоте вращения приводного двигателя) или изменяющейся по известному закону (для регулируемых насосов или при изменении частоты вращения).

Эти допущения позволяют использовать относительно простые математические модели для анализа работы гидропривода и получения удовлетворительных для инженерной практики результатов.

1.2 Классификация и назначение элементов объемного гидропривода

Объемный гидропривод представляет собой совокупность устройств, предназначенных для передачи механической энергии и преобразования движения посредством рабочей жидкости под давлением.

1.2.1 Насосы

Насос – это гидравлическая машина, преобразующая механическую энергию приводного двигателя в энергию потока рабочей жидкости. В объемных гидроприводах применяются преимущественно объемные насосы, в которых подача жидкости осуществляется за счет периодического изменения объема рабочих камер.

Шестеренные насосы просты по конструкции, надежны, имеют невысокую стоимость. Характеризуются постоянным рабочим объемом. Применяются в системах с невысоким и средним давлением.

Аксиально-поршневые насосы обладают высоким КПД, способны работать при высоких давлениях, могут быть как регулируемые (с изменением рабочего объема), так и нерегулируемыми. Более сложны и дороги по сравнению с шестеренными.

Радиально-поршневые насосы применяются в системах с высокими давлениями и большими подачами. Могут быть регулируемые.

Пластинчатые (шиберные) насосы могут быть однократного и двукратного действия, регулируемые и нерегулируемые. Используются в системах со средним давлением.

1.2.2 Гидродвигатели

Гидродвигатель – это гидравлическая машина, преобразующая энергию потока рабочей жидкости в механическую энергию на выходном звене.

Гидроцилиндры обеспечивают возвратно-поступательное движение исполнительного органа. Различают гидроцилиндры одностороннего и двустороннего действия, с односторонним или двусторонним штоком, плунжерные, телескопические.

Гидромоторы обеспечивают вращательное движение выходного вала. По конструкции аналогичны объемным насосам (шестеренные, аксиально-поршневые, радиально-поршневые и др.). Могут быть высокомоментными (низкооборотными) и низкомоментными (высокооборотными).

1.2.3 Гидроаппаратура

Гидравлическая аппаратура является неотъемлемой и критически важной частью любой современной гидросистемы. Основная задача этих сложных технических устройств заключается в обеспечении всестороннего и точного контроля над потоком рабочей жидкости. Это включает в себя возможность изменять его направление, регулировать величину давления и управлять объемным расходом. Эффективность и безопасность работы всей гидравлической системы во многом определяется качеством и правильностью функционирования именно гидроаппаратуры.

Одним из ключевых типов гидроаппаратуры являются гидравлические распределители. Их главная функция – целенаправленно изменять маршрут движения потока рабочей жидкости, направляя его к соответствующим рабочим полостям гидродвигателей (таких как гидроцилиндры или гидромоторы) или к

другим гидравлическим устройствам в системе. Это позволяет управлять движением исполнительных механизмов, включая их запуск, остановку или реверсирование. Для обеспечения гибкости применения, управление гидрораспределителями может осуществляться различными способами: вручную оператором, механически от других частей машины, с помощью электрических сигналов (например, от электромагнитов), гидравлически (давлением самой рабочей жидкости) или пневматически (сжатым воздухом).

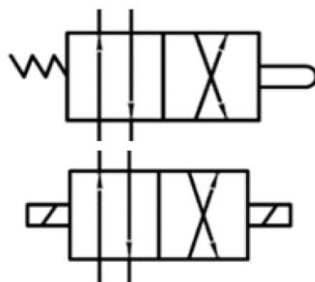


Рисунок 1.1 – Условные обозначения распределителей

Клапаны давления:

- Предохранительные клапаны ограничивают максимальное давление в гидросистеме или ее части, сбрасывая избыток жидкости в бак.
- Переливные клапаны поддерживают заданное давление на входе путем непрерывного слива части жидкости.
- Редукционные клапаны поддерживают на выходе более низкое и постоянное давление по сравнению с давлением на входе
- Обратные клапаны пропускают поток жидкости только в одном направлении.
- Тормозные клапаны, гидрозамки используются для фиксации гидродвигателя в определенном положении или для управления его движением под действием внешней нагрузки.

Регуляторы потока (расхода) и дроссели изменяют величину потока рабочей жидкости за счет изменения проходного сечения. Дроссели создают местное сопротивление потоку. Регуляторы потока поддерживают заданный расход независимо от перепада давлений.

Делители и сумматоры потока делители разделяют один поток на два или более, сумматоры объединяют несколько потоков в один.

Фильтры предназначены для очистки рабочей жидкости от механических примесей, что обеспечивает долговечность и надежность работы гидросистемы. Устанавливаются во всасывающих, напорных или сливных магистралях.

1.2.4 Вспомогательные устройства

Гидробаки служат резервуаром для рабочей жидкости, обеспечивают ее охлаждение, отделение воздуха и оседание загрязнений.

Трубопроводы (гидролинии) соединяют различные элементы гидропривода. Различают всасывающие, напорные, сливные и дренажные линии. Изготавливаются из стальных труб, медных труб или гибких рукавов высокого давления (РВД).

Соединительная арматура (фитинги, штуцеры, тройники, коллекторы) обеспечивают герметичное соединение элементов гидросистемы. (Пример: модуль «Тройник»).

Теплообменники применяются для принудительного охлаждения рабочей жидкости в гидросистемах с высоким тепловыделением.

Гидроаккумуляторы накапливают энергию рабочей жидкости под давлением и отдают ее в систему при необходимости. Используются для сглаживания пульсаций давления, компенсации утечек, питания системы при кратковременных пиковых нагрузках или аварийном отключении насоса.

1.3 Определение мощности гидропривода и выбор насоса

Полезная мощность гидродвигателя – это механическая мощность, развиваемая им на выходном звене.

Для гидродвигателя возвратно-поступательного действия (гидроцилиндра) полезная мощность ($N_{\text{гдв.ц}}$) определяется усилием на штоке и скоростью его движения:

$$N_{\text{гдв.ц}} = F \cdot V \quad (1.1)$$

где:

- $N_{\text{гдв.ц}}$ – полезная мощность гидроцилиндра, Вт;
- F – усилие на штоке, Н;
- V – скорость движения штока, м/с.

Для гидродвигателя вращательного действия (гидромотора) полезная мощность ($N_{\text{гдв.ц}}$) определяется крутящим моментом на валу и его угловой скоростью:

$$N_{\text{гдв.м}} = M \cdot \omega_{\text{м}} \quad (1.2) \text{ или } N_{\text{гдв.ц}} = M \cdot 2\pi \cdot n_{\text{м}} \quad (1.2a)$$

где:

- $N_{\text{гдв.м}}$ – полезная мощность гидромотора, Вт (или кВт);
- M – крутящий момент на валу гидромотора, Н·м;
- $\omega_{\text{м}}$ – угловая скорость вращения вала гидромотора, рад/с;
- $n_{\text{м}}$ – частота вращения вала гидромотора, об/мин.

При работе гидропривода в циклическом режиме полезная мощность определяется как средняя за цикл.

Для гидроцилиндра:

$$N_{\text{гдв.ср.ц}} = \sum F_i \cdot V_i \cdot \Delta t_i / T_{\text{ц}} \quad (1.3)$$

Для гидромотора:

$$N_{\text{гдв.ср.м}} = \sum M_i \cdot \omega_i \cdot \Delta t_i / T_{\text{ц}} \quad (1.4)$$

где:

- $N_{\text{гдв.ср.ц}}$ – средняя полезная мощность за цикл, Вт;
- F_i, M_i – усилие или момент на i -й операции цикла, Н или Н·м;
- V_i, ω_i – скорость или угловая скорость на i -й операции цикла, м/с или рад/с;
- Δt_i – продолжительность i -й операции цикла, с;
- $T_{\text{ц}}$ – общее время цикла, с.

Если в гидросистеме одновременно работает несколько гидродвигателей, то общая полезная мощность гидропривода равна сумме мощностей всех одновременно работающих гидродвигателей.

1.3.2 Определение полезной мощности насоса с учетом коэффициентов запаса

Полезная мощность, которую должен развивать насос ($N_{\text{нп}}$), определяется исходя из полезной мощности гидродвигателей с учетом потерь энергии при ее передаче. Эти потери учитываются введением коэффициентов запаса по усилию (или моменту) и по скорости:

$$N_{\text{нп}} = k_{\text{зу}} \cdot k_{\text{зс}} \cdot N_{\text{гдв}} \quad (1.5)$$

где:

- $N_{\text{нп}}$ – полезная мощность насоса, кВт;
- $k_{\text{зу}}$ – коэффициент запаса по усилию (1,1...1,2), учитывает гидравлические потери давления в гидролиниях и местных сопротивлениях, а также потери на преодоление инерционных сил и трения;
- $k_{\text{зс}}$ – коэффициент запаса по скорости (1,1...1,3), учитывает объемные утечки рабочей жидкости в элементах гидропривода;
- $N_{\text{гдв}}$ – суммарная полезная мощность гидродвигателей, кВт.

Меньшие значения коэффициентов запаса выбираются для гидроприводов, работающих в легком и среднем режимах, а большие – в тяжелом и весьма тяжелом режимах эксплуатации.

1.3.3 Выбор насоса по номинальному давлению и рабочему объему

Зная необходимую полезную мощность насоса, и учитывая, что $N_{\text{нп}} = P_{\text{ном}} \cdot Q_{\text{н}}$ (где $P_{\text{ном}}$ – номинальное давление насоса, $Q_{\text{н}}$ – его подача), можно определить требуемую подачу насоса:

$$Q_{\text{н}} = \frac{N_{\text{нп}}}{P_{\text{ном}}} \quad (1.6)$$

где:

- $Q_{\text{н}}$ – требуемая подача насоса, м³/с (в расчетах часто л/мин, необходим перевод);
- $P_{\text{ном}}$ – номинальное рабочее давление насоса, Па.

Либо, если задана частота вращения приводного вала насоса $n_{\text{н}}$ (об/с или об/мин), можно определить требуемый рабочий объем насоса $q_{\text{н}}$:

$$q_{\text{н}} = \frac{N_{\text{нп}}}{P_{\text{ном}} \cdot n_{\text{н}}} \quad (1.7)$$

- $q_{\text{н}}$ – рабочий объем насоса, м³ (или см³/об);
- $n_{\text{н}}$ – частота вращения вала насоса, с⁻¹ (об/с). Если задана в об/мин, необходимо делить на 60.

Выбор конкретного типоразмера насоса производится из каталогов производителей по двум основным параметрам: номинальному давлению $P_{\text{ном}}$, которое должно быть не меньше расчетного давления в системе, и рабочему объему

q_n (или подаче Q_n при номинальной частоте вращения), ближайшему большему к расчетному значению.

Частота вращения вала насоса зависит от типа приводного двигателя. Для мобильных машин с дизельными двигателями это обычно 1500-2200 об/мин. Для насосов с приводом от электродвигателей номинальные частоты вращения стандартизованы (ГОСТ 12446-80): 480, 600, 750, 960, 1200, 1500, 1920, 2400, 3000 об/мин и др.

После выбора насоса по каталогу, его действительная подача $Q_{нд}$ уточняется с учетом его объемного КПД $\eta_{об}$ и действительной частоты вращения $n_{нд}$:

$$Q_{нд} = q_{нд} \cdot n_{нд} \cdot \eta_{об} \quad (1.8)$$

– $q_{нд}$ – действительный рабочий объем выбранного насоса, м³/об;
 – $n_{нд}$ – действительная частота вращения вала насоса, об/с;
 – $\eta_{об}$ – объемный КПД насоса (из паспортных данных или справочников, например, для шестеренных 0,75–0,9; аксиально-поршневых 0,94–0,97).

Если расчетный рабочий объем насоса получается слишком большим, возможно применение нескольких насосов, установленных параллельно.

1.4 Расчет и выбор гидроцилиндров

Гидроцилиндры являются исполнительными механизмами гидропривода, преобразующими энергию потока рабочей жидкости в механическую энергию поступательного движения штока. Расчет и выбор гидроцилиндра производятся на основе заданных усилий, скоростей и хода штока.

1.4.1 Определение диаметров поршня и штока

Ключевыми геометрическими характеристиками, определяющими конструктивные и эксплуатационные возможности любого гидроцилиндра, являются его основные внутренние размеры: в первую очередь, это диаметр поршня (обозначаемый как D) и, при его наличии, диаметр штока (обозначаемый как d). Процесс точного определения этих фундаментальных параметров не является произвольным, а напрямую зависит от целого ряда факторов. Среди них наиболее существенными являются: направление, в котором прикладывается основное рабочее усилие (то есть, происходит ли выдвижение штока из гильзы цилиндра или, наоборот, его втягивание), а также величины давлений рабочей жидкости, которые создаются или поддерживаются в соответствующих рабочих полостях гидроцилиндра – поршневой и, если применимо, штоковой.

Рассмотрим гидроцилиндр двустороннего действия с односторонним штоком.

При выдвижении штока (рабочая жидкость подается в поршневую полость А (смотреть рис. 1.2).

Уравнение равновесия сил, действующих на шток (без учета инерции):

$$P_1 \cdot F_1 = P_2 \cdot F_2 + R + T \quad (1.9)$$

где:

– P_1 – давление в поршневой (не штоковой) полости, Па;
 – F_1 – площадь поршня, м²; $F_1 = \frac{\pi D^2}{4}$;

- P_2 – давление в штоковой полости, Па;
- F_2 – площадь поршня со стороны штоковой полости, м^2 ; $F_2 = \frac{\pi(D^2 - d^2)}{4}$;
- R – полезное усилие на штоке, Н;
- T – сила трения в уплотнениях и направляющих, Н. Силу трения T можно ориентировочно принять равной $(0,01...0,02)R$.

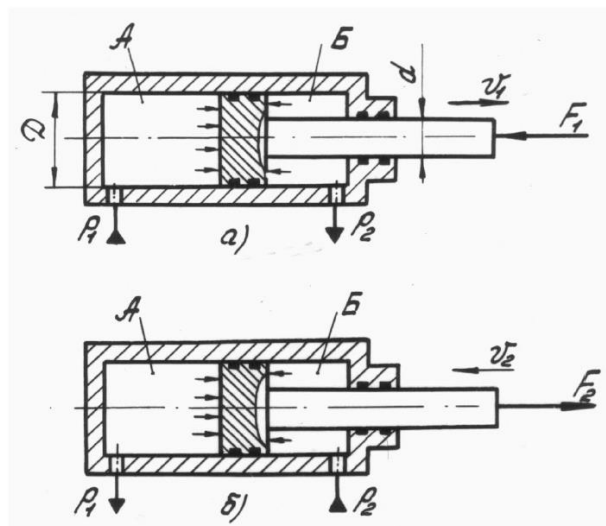


Рисунок 1.2 – Расчетные схемы гидроцилиндров: а) – с поршневой рабочей полостью; б) – со штоковой рабочей полостью

Давления P_1 и P_2 определяются с учетом потерь в гидрелиниях и гидроаппаратуре:

$$P_1 = P_n - \Delta P_{\text{зол}1} - \Delta P_1 \quad (\text{давление в поршневой полости})$$

$P_2 = \Delta P_{\text{зол}2} + \Delta P_2 + \Delta P_{\text{др}} + \Delta P_{\text{ф}}$ (давление в штоковой полости при наличии дросселя и фильтра на сливе)

где P_n – давление, развиваемое насосом, $\Delta P_{\text{зол}1}, \Delta P_{\text{зол}2}$ – перепады давлений на гидрораспределителе, $\Delta P_1, \Delta P_2$ – перепады давлений в напорной и сливной гидрелиниях, $\Delta P_{\text{др}}$ – перепад на дросселе, $\Delta P_{\text{ф}}$ – перепад на фильтре. (Это общая форма, конкретные компоненты потерь зависят от схемы).

Для предварительного расчета, задавшись коэффициентом соотношения диаметров $\varphi = d/D$ (обычно $\varphi = 0,3...0,7$, диаметр поршня D можно определить по формуле:

$$D = \sqrt{\left(\frac{4(R+T)}{\pi \cdot (P_1 - P_2 \cdot (1 - \varphi^2))} \right)} \quad (1.10)$$

Если принять, что давление в штоковой полости P_2 близко к нулю (свободный слив), то формула упрощается.

При втягивании штока (рабочая жидкость подается в штоковую полость Б): $P'_2 \cdot F_2 = P'_1 \cdot F_1 + R' + T$, где P'_2 – давление в штоковой полости (теперь напорной), P'_1 – давление в поршневой полости (теперь сливной), R' – полезное усилие на втягивание.

Полученные расчетные значения диаметров D и d округляются до ближайших стандартных значений по ГОСТ 12447–80. или другим нормативным документам. Стандартные диаметры цилиндров, мм: 25, 32, 40, 50, 63, 80, 100, 110, 125, 140, 160, 180, 200 и т.д.

Толщина стенки гидроцилиндра δ определяется из условия прочности по формуле Ляме:

$$\delta = \left(\frac{D}{2}\right) \cdot \left(\sqrt{\frac{([\sigma] + 0,4P_H)}{([\sigma] - 1,3P_H)}} - 1\right) \quad (1.11)$$

где $[\sigma]$ – допускаемое напряжение на растяжение для материала гильзы.

При $D/\delta \geq 16$ можно использовать упрощенную формулу:

$$\delta = \frac{(P_H \cdot D)}{(2 \cdot [\sigma]) \cdot k_3} \quad (1.12)$$

где k_3 – коэффициент запаса прочности (1,25–2,5).

1.4.2 Расчет скоростей и расходов

Скорость движения штока $v_{пр}$ (при рабочем ходе) и $v_{пх}$ (при холостом/обратном ходе) определяется заданными временами t_p и t_x и ходом поршня S :

$$v_{пр} = S/t_p \quad (1.13)$$

$$v_{пх} = S/t_x \quad (1.14)$$

Расход жидкости $Q_{ц1}$, поступающей в поршневую (не штоковую) полость для обеспечения скорости $v_{пр}$:

$$Q_{ц1} = v_{пр} \cdot F_1 = v_{пр} \cdot \left(\frac{\pi D^2}{4}\right) \quad (1.15)$$

Расход жидкости $Q_{ц2}$, поступающей в штоковую полость для обеспечения скорости $v_{пх}$ (если это рабочий ход):

$$Q_{ц2} = v_{пх} \cdot F_2 = v_{пх} \cdot \left(\frac{\pi(D^2 - d^2)}{4}\right) \quad (1.16)$$

Если расход, подаваемый насосом Q_H , одинаков для прямого и обратного хода, то скорости будут обратно пропорциональны площадям:

$$v_{пр}/v_{пх} = F_2/F_1 = \frac{(D^2 - d^2)}{D^2} = 1 - \varphi^2 \quad (1.17)$$

Действительная скорость штока V_d уточняется после выбора насоса и определения его действительной подачи $Q_{нд}$ с учетом объемных потерь в гидроцилиндре $\Delta Q_{ц}$ и других элементах на пути к цилиндру ($\Delta Q_{зол1}$, $\Delta Q_{пк}$ и т.д.), а также объемного КПД самого цилиндра $\eta_{о.ц}$:

$$V_{д. пр} = \frac{(Q_{нд} - \Delta Q_{зол1} - \Delta Q_{пк1} - \Delta Q_{ц1})}{F_1} \cdot \eta_{о.ц} \quad (1.18)$$

Утечки в силовом гидроцилиндре $\Delta Q_{ц}$ зависят от давления и зазоров, их значения можно найти в справочной литературе.

1.5 Расчет и выбор гидромоторов

Гидромоторы преобразуют энергию потока рабочей жидкости во вращательное движение выходного вала. Их расчет и выбор производятся на основе требуемого крутящего момента на валу и необходимой частоты вращения.

Основные параметры и принцип выбора:

- Крутящий момент (M) определяется нагрузкой на исполнительном механизме, который приводится в движение гидромотором. Если между гидромотором и исполнительным механизмом есть редуктор, то момент на валу гидромотора будет отличаться от момента на рабочем органе с учетом передаточного числа и КПД редуктора.

- Частота вращения (n_m) задается технологическими требованиями к работе исполнительного механизма.

- Рабочий объем (q_m) это объем жидкости, необходимый для одного оборота вала гидромотора. Является ключевой конструктивной характеристикой.

- Давление (P_m) перепад давлений на входе и выходе гидромотора, необходимый для создания требуемого крутящего момента.

- Расход (Q_m) количество жидкости, проходящее через гидромотор в единицу времени для обеспечения заданной частоты вращения.

- КПД (η_m , $\eta_{об.м}$, $\eta_{гм.м}$) объемный, гидромеханический и полный КПД гидромотора, учитывающие внутренние утечки и механические потери.

Методика выбора:

- Определение требуемых параметров на валу гидромотора. Если есть редуктор, то заданный момент и частота вращения рабочего органа пересчитываются на вал гидромотора с учетом передаточного числа редуктора и его КПД.

- Предварительный расчет рабочего объема гидромотора. Зная требуемый крутящий момент и расчетный перепад давлений на гидромоторе (с учетом потерь в системе и КПД гидромотора), можно оценить необходимый рабочий объем. $q_m \approx \frac{(2\pi \cdot M)}{(\Delta P_{расч} \cdot \eta_{гм.м})}$, где $\eta_{гм.м}$ – гидромеханический КПД гидромотора.

- Выбор типоразмера гидромотора. По каталогам производителей подбирается гидромотор с ближайшим большим рабочим объемом и подходящим номинальным давлением. Учитываются также диапазоны рабочих частот вращения.

- Уточнение параметров. Действительный крутящий момент рассчитывается на основе выбранного рабочего объема, фактического перепада давлений и гидромеханического КПД. Действительная частота вращения определяется действительным расходом жидкости через гидромотор (с учетом подачи насоса и объемных утечек в системе) и объемным КПД гидромотора. $Q_{м.действ} = Q_{нд} - \Sigma \Delta Q_{утечек_до_мотора}$. $n_{м.действ} = (Q_{м.действ} / q_{м.выбр}) \cdot \eta_{об.м}$.

- Проверка соответствия. Сравниваются расчетные (действительные) параметры с заданными. Отклонения не должны превышать допустимые пределы (обычно $\pm 10\%$). Если условия не выполняются, рассматривается другой типоразмер гидромотора или корректируются параметры редуктора (если он есть).

Типы гидромоторов:

– Аналогично насосам, гидромоторы бывают шестеренными, аксиально-поршневыми, радиально-поршневыми и др. Выбор типа зависит от требуемых характеристик:

– Низкомоментные (высокооборотные) гидромоторы (часто аксиально-поршневые, шестеренные) используются, как правило, с редукторами для увеличения крутящего момента на рабочем органе.

– Высокомоментные (низкооборотные) гидромоторы (часто радиально-поршневые) могут напрямую соединяться с рабочим органом без редуктора.

Справочные данные по гидромоторам (рабочие объемы, номинальные давления, КПД, диапазоны частот вращения) приводятся в каталогах производителей и специализированной литературе.

1.6 Определение внутреннего диаметра гидролиний и скоростей движения жидкости

Правильный выбор диаметров гидролиний (трубопроводов и рукавов высокого давления) имеет важное значение для минимизации гидравлических потерь, предотвращения кавитации на всасывании насоса и обеспечения стабильной работы гидросистемы.

Основным критерием для выбора диаметра гидролинии является допустимая скорость движения рабочей жидкости. Слишком высокая скорость приводит к значительным потерям давления на трение и в местных сопротивлениях, а также к повышенному шуму и вибрации. Слишком низкая скорость ведет к неоправданному увеличению габаритов и массы трубопроводов.

Рекомендуемые скорости движения рабочей жидкости зависят от назначения гидролинии и рабочего давления в системе:

– Всасывающие линии: 0,5 – 1,5 м/с (для предотвращения кавитации скорость должна быть минимальной).

– Сливные линии (открытые системы): 2 – 3 м/с (иногда до 4 м/с).

– Напорные линии: Скорость зависит от давления: до 5 МПа: 3 – 4 м/с, до 10 МПа: 4 – 5 м/с, до 20 МПа: 5 – 6 м/с, выше 20 МПа: до 7 м/с и более (требует тщательного обоснования).

Методика расчета:

– Определение расчетного расхода жидкости $Q_{нд}$ через данный участок гидролинии (обычно это действительная подача насоса или ее часть, если есть делители потока или параллельные ветви).

– Выбор рекомендуемой скорости для данного типа гидролинии $V_{ж}$.

– Расчет требуемого внутреннего диаметра: $d_p = \sqrt{4 \cdot Q_{нд} / (\pi \cdot V_{ж})}$

– При этом необходимо следить за согласованием единиц измерения (например, если $Q_{нд}$ в л/мин, а $V_{ж}$ в м/с, нужны переводные коэффициенты, как в указанных формулах).

– Выбор стандартного трубопровода. По расчетному диаметру d_p из таблиц стандартных размеров труб (ГОСТ 8732-78, ГОСТ 8734-75 для стальных бесшовных

труб, ГОСТ 16516-80 для условных проходов) выбирается ближайший больший или равный внутренний диаметр d .

– Уточнение действительной скорости жидкости ($V_{ж}$) в выбранном трубопроводе: $V_{жд} = \frac{(4 \cdot Q_{нд})}{(\pi \cdot d^2)}$. Эта скорость используется в дальнейших расчетах потерь давления.

Толщина стенки трубопровода выбирается конструктивно или рассчитывается из условия прочности, особенно для напорных линий высокого давления.

1.7 Расчет потерь давления в гидросистемах и гидроаппаратуре

Движение рабочей жидкости по гидросистеме неизбежно сопровождается потерями энергии, которые проявляются в виде потерь давления. Эти потери необходимо учитывать, так как они снижают давление, достигающее до гидродвигателей, и влияют на общую эффективность гидропривода.

Потери давления можно условно разделить на три основные группы:

– Потери давления на трение в трубопроводах возникают из-за вязкости жидкости и трения ее слоев друг о друга и о стенки трубопровода при движении.

– Потери давления в местных сопротивлениях обусловлены изменением формы, размеров или направления потока в различных элементах гидросистемы (колена, тройники, внезапные сужения/расширения, входы/выходы из емкостей и т.д.).

– Потери давления в гидроаппаратуре – это потери, возникающие при прохождении жидкости через гидрораспределители, клапаны, фильтры и другие управляющие и вспомогательные устройства.

1.7.1 Потери давления на трение в трубопроводах

Величина этих потерь зависит от длины и диаметра трубопровода, скорости потока, плотности и вязкости жидкости, а также от шероховатости внутренней поверхности трубы.

Методика расчета:

– Сначала определяется режим течения жидкости (ламинарный или турбулентный) с помощью числа Рейнольдса (Re). Это безразмерное число зависит от скорости потока, диаметра трубы и кинематической вязкости жидкости.

– В зависимости от режима течения и относительной шероховатости трубы (отношение абсолютной шероховатости к диаметру) определяется коэффициент гидравлического трения (λ). Для ламинарного режима λ зависит только от числа Рейнольдса. Для турбулентного режима – от числа Рейнольдса и шероховатости (используются формулы Блазиуса, Альтшуля или графики Муди).

– Зная λ , длину, диаметр, плотность жидкости и ее скорость, потери давления на трение рассчитываются по формуле Дарси-Вейсбаха.

1.7.2 Потери давления в местных сопротивлениях

Эти потери пропорциональны скоростному напору (плотность, умноженная на квадрат скорости, деленный на два) и коэффициенту местного сопротивления (ξ). Значения коэффициентов ξ для различных типов местных сопротивлений (повороты, сужения, расширения, арматура и т.д.) приводятся в справочной литературе.

Для ламинарного режима течения в расчет потерь на местных сопротивлениях также вводится поправочный коэффициент (b-коэффициент), зависящий от числа Рейнольдса.

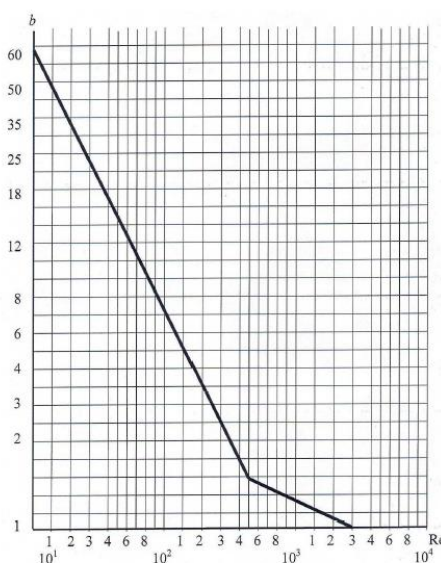


Рисунок 1.3 – График b-коэффициента (поправочного коэффициента для потерь в местных сопротивлениях при ламинарном режиме)

1.7.3 Потери давления в гидроаппаратуре

Потери давления в гидрораспределителях, клапанах, фильтрах и других устройствах обычно указываются в их технических характеристиках (паспортах или каталогах) для номинального расхода. Если фактический расход через аппарат отличается от номинального, перепад давления пересчитывается, как правило, пропорционально квадрату отношения расходов.

Суммарные потери давления в гидролинии или ее участке определяются сложением потерь на трение и всех местных потерь (включая потери в гидроаппаратуре, установленной на этом участке). Для предварительных расчетов, когда детальная информация о местных сопротивлениях отсутствует, их влияние может быть учтено увеличением длины трубопровода на некоторую эквивалентную величину или использованием общего коэффициента запаса.

1.8 Определение КПД гидропривода

Коэффициент полезного действия (КПД) гидропривода является важнейшим показателем его энергетической эффективности. Он показывает, какая часть подведенной к насосу энергии преобразуется в полезную работу на выходном звене гидродвигателя.

Общий (полный) КПД гидропривода (η) является произведением трех основных составляющих: $\eta = \eta_{\text{г}} \cdot \eta_{\text{м}} \cdot \eta_{\text{об}}$ или, что то же самое, произведением гидромеханического и объемного КПД: $\eta = \eta_{\text{гм}} \cdot \eta_{\text{об}}$, где:

- Гидравлический КПД ($\eta_{\text{г}}$) учитывает потери давления в гидролиниях и гидроаппаратуре. Чем меньше эти потери, тем выше гидравлический КПД. Он определяется как отношение давления на гидродвигателе к давлению, развиваемому насосом, за вычетом потерь.

- Механический КПД ($\eta_{\text{м}}$) учитывает потери на трение в подвижных частях насоса, гидродвигателей и других механических элементов гидросистемы. Для системы в целом это произведение механических КПД всех последовательно соединенных элементов (насоса, распределителя, гидродвигателя и т.д.).

- Объемный КПД ($\eta_{\text{об}}$) учитывает утечки рабочей жидкости (объемные потери) через зазоры и уплотнения в насосе, гидродвигателях и гидроаппаратуре. Для системы это произведение объемных КПД всех ее элементов.

- Гидромеханический КПД ($\eta_{\text{гм}}$) объединяет гидравлические и механические потери ($\eta_{\text{гм}} = \eta_{\text{г}} \cdot \eta_{\text{м}}$).

Значения КПД для отдельных элементов (насосов, гидромоторов, распределителей) обычно берутся из их паспортных данных или справочной литературы. Для гидроприводов строительных и дорожных машин общий КПД обычно находится в пределах 0,6 – 0,75.

1.9 Тепловой расчет гидропривода

В процессе работы гидропривода часть энергии теряется и преобразуется в тепло, что приводит к нагреву рабочей жидкости и элементов системы. Перегрев жидкости снижает ее вязкость, ухудшает смазывающие свойства и может привести к отказам уплотнений и выходу гидропривода из строя. Тепловой расчет необходим для определения установившейся температуры рабочей жидкости и, при необходимости, для выбора средств охлаждения (гидробака достаточного объема, теплообменника).

Основные этапы теплового расчета:

- Определение количества выделяемого тепла (G или Q_1): Тепловыделение в гидросистеме пропорционально потерям мощности, то есть разнице между мощностью, потребляемой насосом, и полезной мощностью на гидродвигателях ($N_{\text{пр}} - N_{\text{пос}}$). Учитываются также коэффициенты, характеризующие режим работы (коэффициент продолжительности работы под нагрузкой $k_{\text{н}}$, коэффициент использования номинального давления $k_{\text{д}}$).

- Определение теплоотдачи гидросистемы (Q_2). Тепло отводится от гидросистемы в окружающую среду через поверхности гидробака, трубопроводов и других элементов. Количество отводимого тепла зависит от общей площади

поверхности теплообмена, разности температур рабочей жидкости и окружающей среды, и коэффициента теплопередачи.

- Расчет установившейся температуры рабочей жидкости ($t_{уст}$). В установившемся режиме количество выделяемого тепла равно количеству отводимого тепла ($Q_1 = Q_2$). Из этого условия определяется температура жидкости.

- Сравнение с допустимой температурой: Установившаяся температура не должна превышать максимально допустимую для выбранной рабочей жидкости и условий эксплуатации (обычно 70-75 °С).

- Выбор объема гидробака и/или теплообменника. Если расчетная температура превышает допустимую, необходимо увеличить площадь теплоотдачи. Этого можно достичь увеличением объема гидробака (и, соответственно, его поверхности) или установкой специального теплообменника (радиатора). Объем гидробака часто выбирают исходя из минутной подачи насоса (например, 1,2–3-минутная подача).

1.10 Выбор рабочей жидкости и ее основные характеристики

Рабочая жидкость (РЖ) в объемном гидроприводе выполняет несколько функций: передает энергию, смазывает трущиеся поверхности, отводит тепло и уносит продукты износа. Правильный выбор РЖ существенно влияет на надежность, долговечность и эффективность гидропривода.

Основные критерии выбора РЖ:

- Вязкость и вязкостно-температурные свойства. Вязкость должна быть оптимальной в рабочем диапазоне температур. Слишком низкая вязкость приводит к увеличению утечек и износу, слишком высокая – к увеличению гидравлических потерь и ухудшению низкотемпературных свойств. Важно, чтобы вязкость мало изменялась при изменении температуры.

- Смазывающие свойства. Жидкость должна обеспечивать надежную смазку трущихся пар для минимизации износа.

- Антикоррозионные свойства. Защита металлических деталей от коррозии.

- Стабильность. Устойчивость к окислению, термическому разложению и образованию отложений при высоких температурах и давлениях.

- Температура застывания и вспышки. Температура застывания должна быть на 10-15 °С ниже минимальной рабочей температуры. Температура вспышки важна с точки зрения пожарной безопасности.

- Совместимость с материалами уплотнений и гидросистемы.

- Стоимость и доступность.

В гидроприводах строительных и дорожных машин чаще всего применяются минеральные масла (например, промышленные И-20А, И-30А, гидравлические МГЕ-46В, АМГ-10, ВМГЗ, или современные масла типа HLP, HVLP).

При выборе РЖ необходимо указать ее марку и основные характеристики, такие как:

- Диапазон рабочих температур.

- Кинематическая вязкость при стандартной температуре (например, при 50 °С или 40 °С).

- Плотность.
- Температура застывания.
- Температура вспышки.

Эти данные необходимы для проведения гидравлических и тепловых расчетов.

1.11 Выводы по разделу

В первой главе были рассмотрены фундаментальные теоретические аспекты, необходимые для проектирования и расчета объемных гидроприводов. Были определены основные допущения, упрощающие инженерные расчеты, и представлена классификация ключевых элементов гидросистемы: насосов, гидродвигателей (цилиндров и моторов), гидроаппаратуры и вспомогательных устройств. Особое внимание было уделено методикам определения основных параметров гидропривода:

- Мощность гидропривода рассчитывается исходя из нагрузок и скоростей на исполнительных органах, с последующим определением требуемой мощности и подачи насоса с учетом коэффициентов запаса.

- Выбор гидроцилиндров и гидромоторов осуществляется на основе требуемых усилий/моментов и скоростей/частот вращения, с последующим уточнением их геометрических и рабочих параметров по стандартным рядам и каталогам.

- Диаметры гидролиний выбираются исходя из рекомендуемых скоростей течения рабочей жидкости для минимизации гидравлических потерь.

- Потери давления, возникающие из-за трения жидкости о стенки трубопроводов и в местных сопротивлениях (включая гидроаппаратуру), являются неотъемлемой частью расчета и влияют на общее давление, развиваемое насосом.

- КПД гидропривода (гидравлический, механический, объемный и полный) служит интегральным показателем его энергетической эффективности.

- Тепловой расчет необходим для обеспечения оптимального температурного режима работы гидросистемы и выбора соответствующего объема гидробака или теплообменника.

- Выбор рабочей жидкости производится с учетом ее вязкостно-температурных, смазывающих, антикоррозионных и других эксплуатационных свойств.

Таким образом, первая глава заложила теоретическую базу, описывающую взаимосвязь между конструктивными параметрами элементов гидропривода, его рабочими характеристиками и условиями эксплуатации. Понимание этих основ необходимо для грамотного проектирования эффективных и надежных гидравлических систем, а также для разработки программного обеспечения, автоматизирующего эти расчеты.

2 Разработка программного комплекса для проектирования и расчета гидравлических схем

Для автоматизации и упрощения процесса проектирования и расчета объемных гидроприводов был разработан специализированный программный комплекс. Данный комплекс позволяет пользователю интерактивно создавать гидравлические схемы, задавать параметры их элементов и получать расчетные характеристики системы, включая гидравлические потери, КПД и тепловой баланс.

2.1 Общее описание архитектуры программного комплекса

Программный комплекс построен по клиент-серверной архитектуре, что обеспечивает разделение логики представления данных и пользовательского интерфейса от вычислительной логики и хранения данных.

Клиентская часть (Frontend) реализована как веб-приложение с использованием библиотеки React. Отвечает за:

- Визуализацию каталога доступных гидравлических модулей.
- Предоставление интерактивного рабочего поля для построения гидравлических схем методом drag-and-drop.
- Отображение и редактирование свойств выбранных модулей.
- Отправку данных собранной схемы на сервер для расчета.
- Отображение результатов расчета, полученных от сервера.
- Взаимодействие с сервером для сохранения и загрузки схем.

Серверная часть (Backend) разработана на платформе Node.js с использованием фреймворка Express. Основные функции:

- Прием данных о гидравлической схеме от клиента.
- Выполнение всех необходимых гидравлических и тепловых расчетов на основе полученных данных и заложенных алгоритмов.
- Формирование и отправка результатов расчета обратно клиенту.
- Обработка запросов на сохранение и загрузку схем из базы данных.

База данных (MongoDB) используется для персистентного хранения созданных пользователями гидравлических схем. MongoDB, как документо-ориентированная NoSQL база данных, хорошо подходит для хранения гибких структур данных, какими являются описания схем с их модулями и соединениями.

2.2.1 Обзор используемых технологий (React, DND)

Для разработки клиентской части была выбрана библиотека JavaScript – React. React позволяет строить сложные пользовательские интерфейсы из независимых, переиспользуемых компонентов. Это упрощает разработку, тестирование и поддержку приложения.

Функциональность перетаскивания элементов (drag-and-drop), необходимая для добавления модулей на схему и их перемещения, реализована с помощью библиотеки React DnD. Она предоставляет набор хуков и компонентов для интеграции возможностей перетаскивания в React-приложения.

Стилизация интерфейса выполнена с использованием каскадных таблиц стилей (CSS).

2.2.2 Компонент каталога модулей

Каталог модулей (Catalog.js) представляет собой панель, содержащую список всех доступных для использования в гидравлической схеме компонентов (насосы, цилиндры, фильтры, трубопроводы и т.д.). Каждый элемент в этом списке является отдельным компонентом Module.js.

```
import React from 'react';
import Module from './Module'; // Импорт компонента отдельного модуля
import './styles/Catalog.css'; // Стили для каталога

const Catalog = () => {
  // Массив, описывающий доступные модули
  const modules = [
    { id: 'engine_d245', name: 'Двигатель (Д245)', type: 'engine_input' },
    { id: 'tank', name: 'Гидробак', type: 'tank_output' },
    { id: 'pump_gns_ap30', name: 'Насос (AP30)', type: 'pump', system: 'gns' },
    { id: 'pump_gru_nsh10', name: 'Насос (НШ10)', type: 'pump', system: 'gru' },
    { id: 'cylinder_znu_c63', name: 'Цилиндр ЗНУ (Ц63)', type: 'cylinder', system: 'gns' },
    { id: 'cylinder_gru_c70', name: 'Цилиндр ГРУ (Ц70)', type: 'cylinder', system: 'gru' },
    { id: 'distributor_rge100', name: 'Распределитель (RGE100)', type: 'distributor', system: 'gns' },
    { id: 'power_block_bpg', name: 'Блок питания (BPG)', type: 'block', system: 'gns' },
    { id: 'hydro_block_gbf', name: 'Гидроблок (ГБФ)', type: 'block', system: 'gru' },
    { id: 'filter_frc12', name: 'Фильтр сливной (FRC12)', type: 'filter', system: 'common' },
    { id: 'pipe', name: 'Трубопровод', type: 'pipe', system: 'common' },
    { id: 'tee_splitter', name: 'Тройник', type: 'splitter', system: 'common' },
    { id: 'collector', name: 'Смеситель (2->1)', type: 'collector', system: 'common' }
  ];

  return (
    <div className="catalog"> {/* Основной контейнер каталога */}
      <div className="module-list"> {/* Контейнер для списка модулей */}
        {/* Динамическое создание элементов каталога на основе массива modules */}
        {modules.map((module) => (
          <Module key={module.id} module={module} />
        ))}
      </div>
    </div>
  );
}
```

```

    </div>
  </div>
  );
};

export default Catalog;

```

Листинг 2.1 – Структура каталога и перечень модулей

В этом коде «modules» — это массив объектов, каждый из которых описывает один гидравлический компонент: его уникальный id (используемый для идентификации типа), отображаемое имя, общий тип (для логики свойств и расчетов) и принадлежность к определенной системе (например, ГНС или ГРУ).

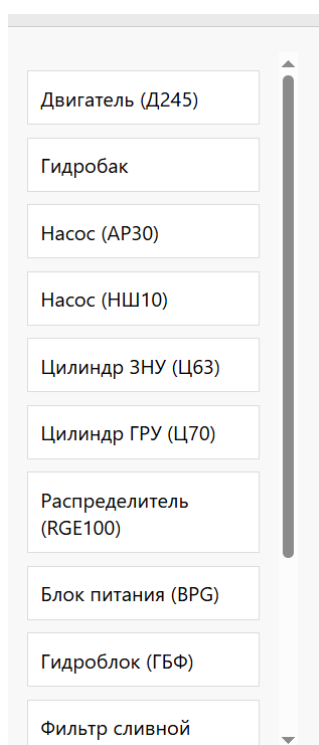


Рисунок 2.1 – Каталог модулей

Для каждого элемента создает экземпляр компонента «Module».

```

import React from 'react';
import { useDrag } from 'react-dnd'; // Хук из React DnD для реализации
перетаскивания

const Module = ({ module }) => {
  // useDrag - это хук, который возвращает объект с состоянием
  перетаскивания
  // и функцию (drag), которую нужно привязать к DOM-элементу, чтобы
  он стал перетаскиваемым.
  const [{ isDragging }, drag] = useDrag(() => ({
    type: 'MODULE', // Уникальный тип для идентификации
    перетаскиваемого элемента (важно для drop-зон)
    item: { // Объект, который будет передан в drop-зону при "падении"
      id: module.id, // ID типа модуля

```

```

    name: module.name,    // Имя модуля
    type: module.type,    // Общий тип модуля
    system: module.system // Система, к которой относится модуль
  },
  collect: (monitor) => ({ // Функция для сбора информации о
    состоянии перетаскивания
      isDragging: !!monitor.isDragging(), // true, если элемент
    сейчас перетаскивается
    }),
  }));

return (
  // Привязываем ref={drag} к div, чтобы сделать его перетаскиваемым
  <div ref={drag} className="module" style={{ opacity: isDragging
? 0.5 : 1 }}>
    {module.name} {/* Отображаем имя модуля */}
  </div>
);
};

export default Module;

```

Листинг 2.2 – Отображение одного элемента в каталоге

Здесь «useDrag» из React DnD инициализируется с указанием типа перетаскиваемого элемента и данных, которые будут ассоциированы с этим элементом во время перетаскивания. Свойство «opacity» изменяется, когда модуль активно перетаскивается, для визуальной обратной связи.

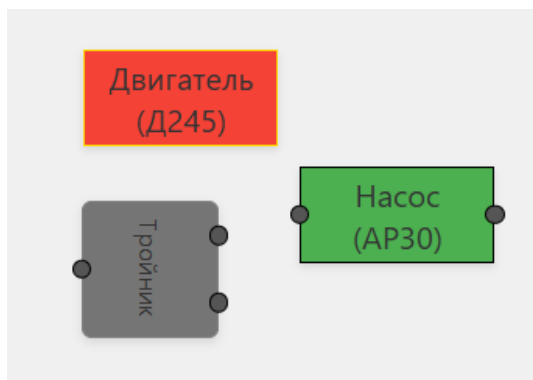


Рисунок 2.2 – Внешний вид модулей

Таким образом, пользователь может выбрать нужный модуль из каталога и перетащить его на основное рабочее поле для построения схемы.

2.2.3 Основное рабочее поле

Компонент «MainScreen.js» является центральной интерактивной областью приложения, предназначенной для визуального конструирования гидравлических схем. Пользователь перетаскивает на это поле модули из каталога, располагает их в

нужных позициях и соединяет между собой, формируя логическую структуру гидропривода.

Основные функции компонента «MainScreen.js»:

- Отрисовка модулей: Компонент отвечает за отображение всех экземпляров гидравлических модулей, добавленных пользователем на схему. Каждый модуль представлен отдельным компонентом «DraggableModule.js», который получает свои уникальные данные (тип, имя, текущие координаты, свойства) и функции обратного вызова для взаимодействия.

- Создание и отображение соединений: Между портами модулей могут быть установлены соединения, которые визуализируются в виде линий со стрелками, указывающими направление потока рабочей жидкости. «MainScreen» хранит информацию об этих соединениях и отрисовывает их с помощью SVG-графики. Координаты для отрисовки линий динамически вычисляются на основе текущего положения соединенных модулей.

- Обработка перетаскивания (Drop Zone): «MainScreen» сконфигурирован как область, принимающая перетаскиваемые модули, с помощью хука «useDrop» из библиотеки React DnD. Когда пользователь перетаскивает новый модуль из каталога и «бросает» его на «MainScreen», компонент определяет его координаты на холсте с учетом текущего масштаба и смещения вида («viewBox») и вызывает функцию обратного вызова «onDrop» для добавления нового экземпляра модуля в общее состояние приложения. Если перемещается уже существующий на холсте модуль, то после «падения» его на новое место вызывается функция «updateModulePosition».

- Панорамирование холста: Для удобной работы с большими схемами реализована функция панорамирования. Пользователь может «схватить» (зажав левую или среднюю кнопку мыши) пустое пространство холста и перемещать его, изменяя видимую область схемы. Это достигается путем отслеживания смещения курсора мыши и соответствующего изменения координат «x» и «y» объекта «viewBox».

- Масштабирование холста: Пользователь может изменять масштаб отображения схемы с помощью колеса мыши. Прокрутка колеса изменяет коэффициент масштабирования «k» в объекте «viewBox». Масштабирование происходит относительно положения курсора мыши.

- Управление состоянием вида («viewBox»): Объект «viewBox», содержащий смещения «x», «y» и коэффициент масштаба «k», управляется в «MainScreen» и передается дочерним компонентам для корректного пересчета координат и размеров.

```
// src/components/MainScreen.js
import React, { useState, useRef, useCallback, useEffect } from
'react';
import { useDrop } from 'react-dnd';
import DraggableModule, {
  MODULE_WIDTH, // и другие константы размеров
  // ...
} from '../DraggableModule';
// ...

const MainScreen = ({
```



```

modules,
connections,
onDrop,
onModuleClick,
updateModulePosition,
onAddConnection,
onRemoveConnection,
selectedModule,
detailedResults,
}) => {
  const mainScreenRef = useRef(null);
  const moduleContainerRef = useRef(null);

  const [viewBox, setViewBox] = useState({ x: 0, y: 0, k: 1 });
  // ... (state для isPanning, panStartPoint, initialViewBox)

  // ... (handleLineClick)

  const [, drop] = useDrop(() => ({
    accept: ['MODULE', 'MODULE_INSTANCE'],
    drop: (item, monitor) => {
      const offset = monitor.getClientOffset();
      const container = moduleContainerRef.current;

      if (offset && container) {
        const containerRect = container.getBoundingClientRect();
        let relativeXToViewport = offset.x - containerRect.left;
        let relativeYToViewport = offset.y - containerRect.top;

        let canvasX = (relativeXToViewport / viewBox.k) + viewBox.x;
        let canvasY = (relativeYToViewport / viewBox.k) + viewBox.y;

        if (!item.instanceId) { // Это новый модуль из каталога
          let dropWidth = MODULE_WIDTH;
          let dropHeight = MODULE_HEIGHT;
          // ... (определение dropWidth, dropHeight для разных типов модулей)
          canvasX -= (dropWidth / 2);
          canvasY -= (dropHeight / 2);
        }
        onDrop(item, { x: canvasX, y: canvasY });
        return { name: 'MainScreen' };
      }
      return undefined;
    },
    // ... (collect функция)
  })), [onDrop, modules, viewBox.x, viewBox.y, viewBox.k]);

```

```

const getPortCenterOnCanvas = (portId, portRole = 'output') => {
  // ... (ключевая логика определения координат порта на холсте)
  // Находит модуль по ID, учитывает его тип для смещения порта,
  // возвращает { x, y } координаты на холсте.
  // Пример для стандартного выходного порта:
  // if (moduleInstance) {
  //   portCanvasOffsetX = currentModuleWidth;
  //   portCanvasOffsetY = currentModuleHeight * 0.5;
  //   return { x: moduleCanvasX + portCanvasOffsetX, y:
moduleCanvasY + portCanvasOffsetY };
  // }
  // ... (полная логика здесь сокращена для примера)
  const portIdStr = String(portId);
  let moduleInstance;
  let isMultiPort = false;
  let portIndex = 0;

  if (portIdStr.includes('_out')) {
    const parts = portIdStr.split('_out');
    moduleInstance = modules.find(m => String(m.instanceId)
=== parts[0]);
    portIndex = parseInt(parts[1], 10);
    isMultiPort = true;
  } else if (portIdStr.includes('_in')) {
    const parts = portIdStr.split('_in');
    moduleInstance = modules.find(m => String(m.instanceId)
=== parts[0]);
    portIndex = parseInt(parts[1], 10);
    isMultiPort = true;
  } else {
    moduleInstance = modules.find(m => String(m.instanceId)
=== portIdStr);
  }

  if (!moduleInstance || !moduleInstance.position) return null;

  const moduleCanvasX = moduleInstance.position.x;
  const moduleCanvasY = moduleInstance.position.y;

  let currentModuleWidth = MODULE_WIDTH;
  let currentModuleHeight = MODULE_HEIGHT;
  // ... (определение currentModuleWidth/Height для разных
типов)

  let portCanvasOffsetX, portCanvasOffsetY;
  if (portRole === 'output') {
    portCanvasOffsetX = currentModuleWidth;
    portCanvasOffsetY = currentModuleHeight * (isMultiPort &&
moduleInstance.type === 'splitter' ? (portIndex === 0 ? 0.25 : 0.75)
: 0.5);
  } else { /* input */
    portCanvasOffsetX = 0;

```

```

        portCanvasOffsetY = currentModuleHeight * (isMultiPort &&
moduleInstance.type === 'collector' ? (portIndex === 0 ? 0.25 : 0.75)
: 0.5);
    }
    return { x: moduleCanvasX + portCanvasOffsetX, y:
moduleCanvasY + portCanvasOffsetY };
};

// ... (useCallback для handleMouseDown, handleMouseMove,
handleMouseUp, handleWheel)
// ... (useEffect для подписки/отписки от событий)

return (
    <div
        ref={el) => { drop(el); mainScreenRef.current = el; }}
        className="main-screen"
        onMouseDown={handleMouseDown}
        // ...
    >
        <div ref={moduleContainerRef} className="module-
container" /* ... */ >
            <div
                className="canvas-layer"
                style={{
                    // ...
                    transform: `scale(${viewBox.k})
translate(${-viewBox.x}px, ${-viewBox.y}px)` ,
                    // ...
                }}
            >
                {modules.map((module) => (
                    <DraggableModule
                        key={module.instanceId}
                        module={module}
                        // ... (пропсы)
                        viewBoxScale={viewBox.k} // Важно для
DraggableModule
                    />
                ))}
                <svg /* ... для отрисовки соединений ... */ >
                    <defs>
                        <marker id="arrow" /* ... */ >
                            <path d="M 0 0 L 10 5 L 0 10 z"
fill="black" />
                        </marker>
                    </defs>
                    {connections.map(conn => {
                        const startCanvas =
getPortCenterOnCanvas(String(conn.sourceId), 'output');
                        const endCanvas =
getPortCenterOnCanvas(String(conn.targetId), 'input');
                        if (startCanvas && endCanvas) {
                            return (

```

```

                                <g                                key={conn.id}
className="connection-line" /* ... */ >
                                <line
                                x1={startCanvas.x}
y1={startCanvas.y}
                                x2={endCanvas.x}
y2={endCanvas.y}
                                stroke="black"
                                strokeWidth={2
viewBox.k} // Адаптивная толщина линии
                                markerEnd="url(#arrow)"
                                onClick={ (e)
handleLineClick(e, conn.id)}
                                // ...
                                />
                                </g>
                                );
                                }
                                return null;
                                }}}
                                </svg>
                                </div>
                                </div>
                                {/* ... (панель результатов) ... */}
                                </div>
                                );
                                };
export default MainScreen;

```

Листинг 2.2 - Фрагмент компонента «MainScreen.js»: инициализация и обработка события «падения» модуля

В данном листинге кода «MainScreen.js» продемонстрирована инициализация компонента и, в частности, хук «useDrop» из React DnD. Этот хук определяет, что «MainScreen» может принимать перетаскиваемые элементы двух типов: «MODULE» (новые модули из каталога) и «MODULE_INSTANCE» (существующие модули, перемещаемые по холсту).

При «падении» элемента (событие «drop»), код получает экранные координаты курсора («monitor.getClientOffset()») и геометрию контейнера модулей («moduleContainerRef.current.getBoundingClientRect()»). На основе этих данных вычисляются относительные координаты «падения» элемента внутри видимой области контейнера. Затем эти координаты преобразуются в систему координат самого холста, который может быть смещен и отмасштабирован. Эта трансформация учитывает текущие значения «viewBox.x», «viewBox.y» (смещение) и «viewBox.k» (масштаб). Если добавляется новый модуль (у него еще нет «instanceId»), его позиция дополнительно корректируется так, чтобы центр модуля оказался под курсором, с учетом стандартных или специфических размеров модуля (например, трубы или тройники имеют отличные от стандартных размеры). После вычисления итоговых координат на холсте вызывается функция «onDrop», переданная из родительского компонента «App.js», которая обрабатывает добавление нового

модуля или обновление позиции существующего в общем состоянии приложения. Возврат { name: 'MainScreen' } в функции «drop» является требованием React DnD для корректной работы механизма перетаскивания.

Функция «getPortCenterOnCanvas», полный код которой присутствует в исходном файле, но здесь опущен для краткости, играет важную роль в вычислении точных координат центров входных и выходных портов каждого модуля. Эти координаты необходимы для правильной отрисовки SVG-линий, представляющих соединения между модулями. Функция учитывает положение модуля на холсте, его тип (который определяет размеры и расположение портов; например, у сплиттеров и коллекторов порты расположены иначе, чем у стандартных модулей), а также текущий масштаб и смещение холста («viewBox»).

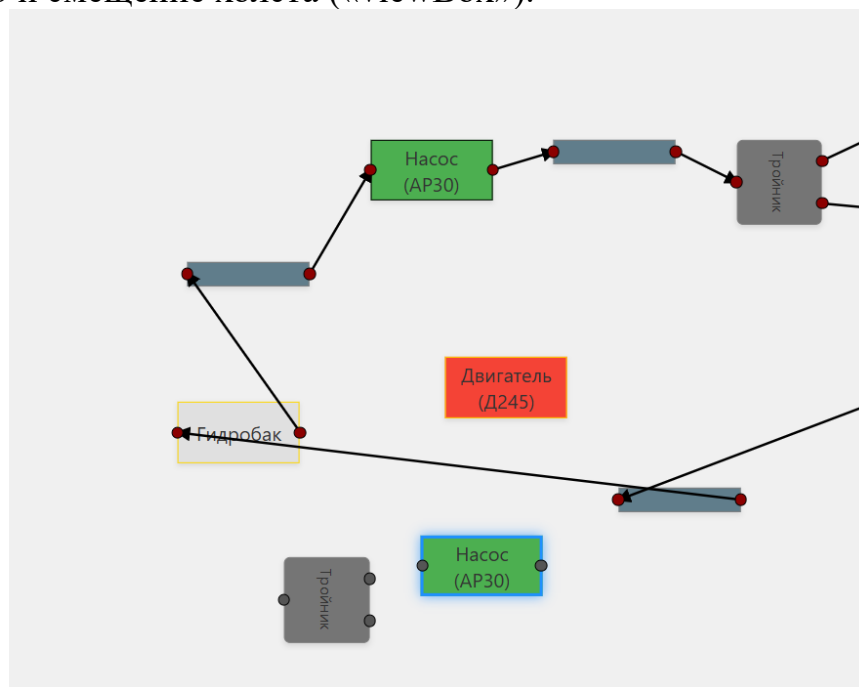


Рисунок 2.3 – Основное рабочее поле

Реализация панорамирования и масштабирования холста осуществляется через обработчики событий мыши («handleMouseDown», «handleMouseMove», «handleMouseUp») и колеса мыши («handleWheel»). Хуки «useCallback» используются для мемоизации этих обработчиков, предотвращая их ненужное пересоздание при каждом рендере компонента. Хук «useEffect» обеспечивает корректное добавление и удаление слушателей этих событий на DOM-элементе «mainScreenRef.current», что предотвращает утечки памяти и гарантирует, что обработчики активны только тогда, когда компонент смонтирован. При панорамировании изменяются координаты «viewBox.x» и «viewBox.y», а при масштабировании – коэффициент «viewBox.k», что и создает эффект перемещения и изменения размера холста.

Визуализация модулей на холсте происходит путем итерации по массиву «modules» (полученному через props) и рендеринга компонента «DraggableModule» для каждого элемента массива. Соединения между модулями отрисовываются в отдельном SVG-слое, который накладывается поверх слоя с модулями. Каждая линия соединения рисуется между рассчитанными центрами соответствующих

портов и имеет маркер в виде стрелки на конце, указывающий направление потока. Толщина линий соединений также адаптируется к текущему масштабу «viewBox.k», чтобы они выглядели адекватно при разном уровне приближения.

2.2.3.1 Реализация перетаскивания модулей («DraggableModule.js»)

Компонент «DraggableModule.js» отвечает за визуальное представление и интерактивное поведение каждого отдельного гидравлического модуля, размещенного на основном рабочем поле «MainScreen». Он инкапсулирует логику, позволяющую пользователю перетаскивать модуль по холсту, а также содержит механизмы для инициации и завершения гидравлических соединений через его входные и выходные порты.

Модуль отображается как прямоугольный блок, внутри которого выводится его наименование. Внешний вид, включая размеры и цвет, может варьироваться в зависимости от типа гидравлического компонента, который он представляет. Например, модули типа «pipe» (трубопровод) имеют меньшую высоту, а модули «splitter» (тройник) и «collector» (смеситель) могут иметь специфическую форму для наглядного отображения нескольких портов. Цвет модуля по умолчанию серый, но может быть изменен пользователем через панель свойств. Выбранный модуль подсвечивается специальной рамкой. Для модулей типа «pipe» также предусмотрено отображение расчетных данных (скорость потока, число Рейнольдса, коэффициент гидравлического трения λ , потери давления) непосредственно на модуле, если эти данные доступны после проведения расчетов сервером. Название модуля и дополнительная информация отображаются только в том случае, если текущий масштаб холста («viewBoxScale») достаточно велик для их комфортного чтения.

Для того чтобы пользователь мог перемещать модули по холсту, используется хук «useDrag» из библиотеки React DnD. При начале перетаскивания модуль идентифицируется типом «MODULE_INSTANCE». В объект «item», ассоциированный с перетаскиваемым элементом, помещаются его уникальный идентификатор («instanceId»), текущие координаты на холсте («initialPosition») и его тип («type»). По завершении операции перетаскивания (событие «end») происходит анализ результата. Если модуль был успешно «брошен» на рабочее поле («MainScreen»), как это определяется функцией «monitor.getDropResult()», то вычисляется смещение модуля относительно его начальной позиции. Это смещение, полученное в экранных координатах («monitor.getDifferenceFromInitialOffset»), пересчитывается в координаты холста с обязательным учетом текущего коэффициента масштабирования «viewBoxScale». Новые, скорректированные координаты передаются в функцию обратного вызова «onUpdatePosition», которая обновляет позицию модуля в общем состоянии приложения («App.js»).

```
// src/components/DraggableModule.js
import React, { useRef, useEffect } from 'react';
import { useDrag, useDrop } from 'react-dnd'; // Импорт хуков для
drag-and-drop

// ... (импорт констант размеров MODULE_WIDTH, MODULE_HEIGHT и т.д.)
```

```

const DraggableModule = ({
  module, // Данные текущего модуля (id, name, type,
position, properties)
  onClick, // Функция обратного вызова для обработки
клика по модулю
  onUpdatePosition, // Функция для обновления позиции модуля в
состоянии App
  onConnect, // Функция для создания нового соединения
connections, // Массив текущих соединений (для проверки
занятости портов)
  isSelected, // Флаг, выбран ли данный модуль
  calculatedPipeData, // Рассчитанные данные для труб (для
отображения)
  viewBoxScale, // Текущий масштаб холста (передается из
MainScreen)
}) => {
  const moduleRef = useRef(null); // Ссылка на DOM-элемент самого
модуля
  const isSplitter = module.type === 'splitter';
  const isCollector = module.type === 'collector';
  const isPipe = module.type === 'pipe';
  let displayTitle = module.name; // Текст для всплывающей подсказки

  // Логика формирования displayTitle для труб с расчетными данными
  if (isPipe && calculatedPipeData && typeof calculatedPipeData ===
'object') {
    // ... (поиск данных для текущей трубы в calculatedPipeData)
    // if (foundPipeInfo) { displayTitle = ... ; }
  }

  // Хук useDrag для реализации перетаскивания всего модуля
  const [{ isDragging: isModuleDragging }, moduleDrag, preview] =
useDrag(() => ({
    type: 'MODULE_INSTANCE', // Тип перетаскиваемого элемента
    item: () => ({ // Данные, ассоциированные с перетаскиваемым
элементом
      instanceId: module.instanceId,
      initialPosition: { x: module.position?.x || 0, y:
module.position?.y || 0 },
      type: module.type,
    }),
    collect: (monitor) => ({
      isModuleDragging: monitor.isDragging(), // Состояние:
перетаскивается ли модуль сейчас
    }),
    end: (item, monitor) => { // Функция, вызываемая при завершении
перетаскивания
      const dropResult = monitor.getDropResult(); // Информация о
том, куда был "брошен" модуль
      const deltaScreen = monitor.getDifferenceFromInitialOffset();
// Смещение в экранных координатах
      const currentScale = viewBoxScale || 1; // Текущий масштаб

```

```

        // Проверка, что модуль "брошен" на MainScreen и есть смещение
        if (item && deltaScreen && dropResult && dropResult.name ===
'MainScreen' && currentScale !== 0) {
            // Пересчет экранного смещения в смещение на холсте
            const deltaCanvasX = deltaScreen.x / currentScale;
            const deltaCanvasY = deltaScreen.y / currentScale;
            // Расчет новых координат модуля на холсте
            const newCanvasX = item.initialPosition.x + deltaCanvasX;
            const newCanvasY = item.initialPosition.y + deltaCanvasY;
            // Вызов функции для обновления позиции модуля в глобальном
состоянии
            onUpdatePosition(item.instanceId, { x: newCanvasX, y:
newCanvasY });
        }
    }, [ // Зависимости хука
        module.instanceId,
        module.position?.x,
        module.position?.y,
        onUpdatePosition,
        viewBoxScale,
    ]);

    // ... (Далее следует логика для портов: useOutputPortDrag и
useInputPortDrop) ...

    // Привязка хуков drag и preview к DOM-элементу модуля
    useEffect(() => {
        if (moduleRef.current) {
            preview(moduleRef.current); // preview для возможности
кастомизации отображения при перетаскивании
            moduleDrag(moduleRef.current); // moduleDrag делает сам DOM-
элемент источником перетаскивания
        }
    }, [preview, moduleDrag]);

    const handleClick = () => {
        onClick(module); // Обработка простого клика по модулю (например,
для его выбора)
    };

    // ... (Определение стилей `styles` для модуля) ...

    return (
        <div
            ref={moduleRef} // Привязка ссылки к основному div модуля
            className={`draggable-module type-${module.type} ${isSelected
? 'selected' : ''}`}
            style={styles}
            onClick={handleClick}
            title={displayTitle} // Всплывающая подсказка
        >
            {/* Отображение имени модуля и его портов */}
            {/* ... (JSX для имени и портов) ... */}

```



```

    </div>
  );
};

// ... (экспорт констант и компонента)
export default DraggableModule;

```

Листинг 2.3 – Фрагмент компонента «DraggableModule.js»: логика перетаскивания модуля

В представленном листинге 2.3 показана основная часть компонента «DraggableModule.js», отвечающая за его перемещение по холсту. Хук «useDrag» из библиотеки React DnD конфигурируется для типа «MODULE_INSTANCE». При завершении перетаскивания (в колбэке «end») происходит проверка, что модуль был "отпущен" над компонентом «MainScreen». Если это так, то смещение модуля, полученное в экранных координатах, корректируется с учетом текущего масштаба «viewBoxScale», переданного из «MainScreen». Это обеспечивает правильное позиционирование модуля на виртуальном холсте независимо от уровня его масштабирования. Затем вызывается функция «onUpdatePosition» для сохранения новых координат модуля. Хук «useEffect» используется для корректной привязки функциональности перетаскивания к DOM-элементу модуля.

2.2.3.2 Механизм соединения модулей (порты, линии связи)

Соединение гидравлических модулей является ключевой операцией при построении схемы. В компоненте «DraggableModule.js» это реализуется через систему входных и выходных портов. Каждый стандартный модуль имеет один входной порт (слева) и один выходной порт (справа). Специализированные модули, такие как «Тройник» (Splitter) и «Коллектор» (Collector), имеют по два выходных или два входных порта соответственно, расположенных на их боковых гранях.

Чтобы пользователь мог инициировать создание соединения, выходные порты модулей сделаны перетаскиваемыми. Для этого используется тот же хук «useDrag» из React DnD, но с другим типом перетаскиваемого элемента – «CONNECTION_PORT».

– Когда пользователь начинает тянуть линию от выходного порта, в перетаскиваемый элемент («item») помещается «sourceId». Этот «sourceId» является уникальным идентификатором порта-источника. Для стандартных модулей это «instanceId» самого модуля. Для модулей с несколькими выходами (например, тройник) идентификатор порта формируется как «module.instanceId + _out0» для первого выхода и «module.instanceId + _out1» для второго.

– Функция «canDrag» в конфигурации «useDrag» проверяет, не занят ли уже данный выходной порт существующим соединением. Перетаскивание разрешено только со свободного порта, что предотвращает создание нескольких исходящих линий из одного и того же выходного порта (за исключением тройников, где каждый выходной порт обрабатывается индивидуально).

Входные порты модулей служат целями, на которые пользователь может бросить линию, начатую от выходного порта другого модуля. Для этого используется хук «useDrop», настроенный на прием элементов типа «CONNECTION_PORT».

Функция «canDrop» в конфигурации «useDrop» выполняет ряд проверок перед тем, как разрешить падение:

- Целевой модуль не должен быть модулем начального типа («engine_input») или конечного типа («tank_output», если это не специальный вход для всасывающей линии насоса), так как в них нельзя "войти" стандартным образом.
- Входной порт, на который пытаются "бросить" соединение, должен быть свободен.
- Запрещено соединять модуль сам с собой (выходной порт модуля не может быть соединен с его же входным портом).

Если все проверки пройдены успешно, и пользователь отпускает линию над допустимым входным портом, вызывается функция «drop». В этой функции происходит вызов колбэка «onAddConnection», которому передаются «sourceId» (из перетаскиваемого элемента) и «targetPortId» (идентификатор входного порта, на который было совершено падение). Этот колбэк отвечает за добавление нового соединения в общее состояние приложения.

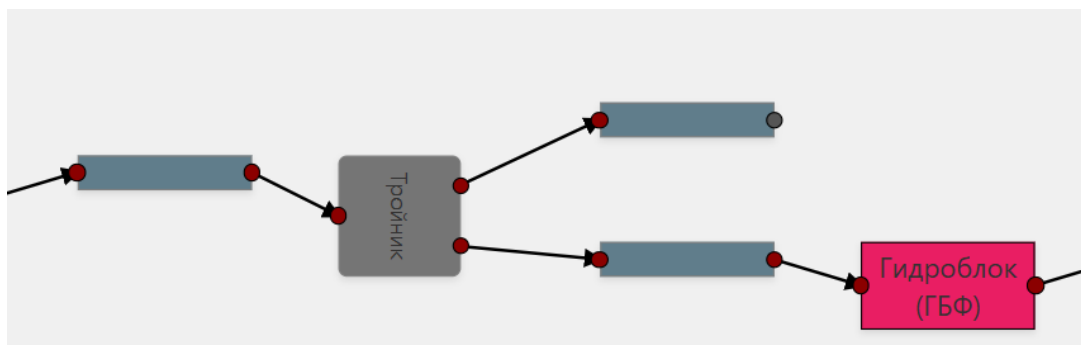


Рисунок 2.4 – Соединение модулей

Для улучшения пользовательского опыта, при наведении перетаскиваемой линии на допустимый входной порт, он может изменять свой внешний вид (например, цвет), сигнализируя о возможности соединения. Это реализуется с помощью флагов «isOver» и «canDrop», возвращаемых хуком «useDrop».

```
const DraggableModule = ({ /* ... пропсы ... */ onConnect,
connections, /* ... */ }) => {
  // ... (ref'ы и логика перетаскивания самого модуля, isSplitter,
isCollector и т.д.)

  // Вспомогательная функция для проверки, занят ли порт
  const getPortBusyState = (portIdentifier, portRole) => {
    if (portRole === 'output') {
      // Проверяем, есть ли соединение, где данный portIdentifier
является источником
      return connections.some(conn => String(conn.sourceId) ===
String(portIdentifier));
    } else { // portRole === 'input'
```

```

        // Проверяем, есть ли соединение, где данный portIdentifier
        // является целью
        return connections.some(conn => String(conn.targetId) ===
String(portIdentifier));
    }
};

// Хук для создания источника перетаскивания из ВЫХОДНОГО порта
const useOutputPortDrag = (portIdentifier, canDragCondition,
itemData = {}) => {
    const [{ isConnecting }, portDrag] = useDrag(() => ({
        type: 'CONNECTION_PORT', // Тип элемента, чтобы входные порты
        // могли его "поймать"
        item: { sourceId: portIdentifier, ...itemData }, // Передаем ID
        // порта-источника
        // Разрешаем перетаскивание только если условие
        // canDragCondition истинно
        // и данный выходной порт еще не используется в другом
        // соединении
        canDrag: canDragCondition && !getPortBusyState(portIdentifier,
'output'),
        collect: (monitor) => ({
            isConnecting: monitor.isDragging(), // Флаг, что от этого
            // порта сейчас тянется линия
        })),
    })), [module.instanceId, portIdentifier, connections,
canDragCondition, itemData]);
    return portDrag; // Возвращаем ref для DOM-элемента порта
};

// Создаем источники перетаскивания для портов
// Для стандартного модуля выходной порт идентифицируется его
instanceId
const defaultPortDrag = useOutputPortDrag(
    module.instanceId,
    !isSplitter && !isCollector && module.type !== 'end' &&
module.type !== 'engine_input',
);
// Для тройника (Splitter) создаем два отдельных источника для его
выходов
const splitterPortDrag0 = useOutputPortDrag(
    `${module.instanceId}_out0`, // Уникальный ID для первого выхода
    isSplitter,
    { outputPortIndex: 0, isSplitterOutput: true, splitterInstanceId:
module.instanceId }
);
const splitterPortDrag1 = useOutputPortDrag(
    `${module.instanceId}_out1`, // Уникальный ID для второго выхода
    isSplitter,
    { outputPortIndex: 1, isSplitterOutput: true, splitterInstanceId:
module.instanceId }
);
// Для коллектора (Collector) выходной порт также стандартный
const collectorPortDrag = useOutputPortDrag(

```

```

module.instanceId,
isCollector, // Условие - это коллектор
{ outputPortIndex: 0 }
);

// Хук для создания цели "падения" на ВХОДНОЙ порт
const useInputPortDrop = (targetPortId, isEnabled = true) => {
  const [{ isOver, canDrop }, portDropRefHook] = useDrop(() => ({
    accept: 'CONNECTION_PORT', // Принимает только элементы типа
'CONNECTION_PORT'
    canDrop: (item) => { // Логика, определяющая, можно ли "бросить"
сюда линию
      if (!isEnabled) return false;
      // Нельзя подключаться к "стартовым" модулям (двигатель)
      if (module.type === 'start' || module.type ===
'engine_input') return false;
      // Нельзя подключаться к уже занятому входному порту
      if (getPortBusyState(targetPortId, 'input')) return false;

      // Предотвращение самосоединения
      let sourceInstanceIdForSelfCheck = item.sourceId;
      // Если источник - это выходной порт сплиттера, берем ID
самого сплиттера для проверки
      if (item.isSplitterOutput) {
        sourceInstanceIdForSelfCheck = item.splitterInstanceId;
      }
      if (String(sourceInstanceIdForSelfCheck) ===
String(module.instanceId)) return false;

      return true; // Если все проверки пройдены, "падение"
разрешено
    },
    drop: (item) => { // Вызывается при успешном "падении"
      if (isEnabled && item.sourceId && onConnect) {
        // Вызываем функцию onConnect для создания нового
соединения
        onConnect(item.sourceId, targetPortId);
      }
    },
    collect: monitor => ({ // Сбор информации о состоянии "падения"
      isOver: isEnabled && monitor.isOver() && monitor.canDrop(),
      canDrop: isEnabled && monitor.canDrop(),
    }),
  })), [module.instanceId, module.type, connections, targetPortId,
onConnect, isEnabled]);
  return [{ isOverInput: isOver, canDropOnInput: canDrop },
portDropRefHook]; // Возвращаем состояние и ref
};

// Создаем цели "падения" для входных портов
// Для стандартного модуля входной порт идентифицируется его
instanceId

```

```

const [{ isOverInput: isOverDefaultIn, canDropOnInput:
canDropDefaultIn }, defaultPortDrop] = useInputPortDrop(
  module.instanceId,
  !isCollector // Коллектор имеет специфичные входные порты
);
// Для коллектора (Collector) создаем две отдельные цели для его
входов
const [{ isOverInput: isOverCollectorIn0, canDropOnInput:
canDropCollectorIn0 }, collectorPortDrop0] = useInputPortDrop(
  `${module.instanceId}_in0`, // Уникальный ID для первого входа
  isCollector
);
const [{ isOverInput: isOverCollectorIn1, canDropOnInput:
canDropCollectorIn1 }, collectorPortDrop1] = useInputPortDrop(
  `${module.instanceId}_in1`, // Уникальный ID для второго входа
  isCollector
);

// ... (useEffect для привязки moduleDrag и preview)
// ... (handleClick)
// ... (стили `styles` и `portBaseStyle`, функции
`finalGetInputPortStyle`, `finalGetOutputPortStyle`)

return (
  <div
    ref={moduleRef}
    // ...
    title={displayTitle}
  >
    /* ... (отображение имени модуля) ... */

    /* Рендеринг входных портов (условно) */
    {module.type !== 'start' && module.type !== 'engine_input' &&
(
  <>
    {isCollector ? (
      <>
        <div
          ref={collectorPortDrop0} // Привязка ref для drop-
зоны порта
          className="port input-port collector-in-0"
          // ... (стили с учетом isOverCollectorIn0,
canDropCollectorIn0)
          title={`Входной порт 1 (${module.name})`}
        />
        <div
          ref={collectorPortDrop1}
          className="port input-port collector-in-1"
          // ... (стили с учетом isOverCollectorIn1,
canDropCollectorIn1)
          title={`Входной порт 2 (${module.name})`}
        />
      </>
    ) : (

```

```

        <div
            ref={defaultPortDrop}
            className="port input-port"
            // ... (стили с учетом isOverDefaultIn,
canDropDefaultIn)
            title={`Входной порт (${module.name})`}
        />
    )}
</>
)}

{/* Рендеринг выходных портов (условно) */}
{module.type !== 'engine_input' && module.type !== 'end' && (
    <>
        {isSplitter ? (
            <>
                <div
                    ref={splitterPortDrag0} // Привязка ref для drag-
источника порта
                    className="port output-port splitter-out-0"
                    // ... (стили)
                    title={`Выходной порт 1 (${module.name})`}
                />
                {/* ... (аналогично для splitterPortDrag1) ... */}
            </>
        ) : (
            <div
                ref={module.type === 'collector' ? collectorPortDrag :
defaultPortDrag}
                className="port output-port"
                // ... (стили)
                title={module.type === 'tank_output' ? "Выход на
всасывание" : `Выходной порт (${module.name})`}
            />
        )}
    </>
)}
</div>
);
};
export default DraggableModule;

```

Листинг 2.4 – Фрагмент компонента «DraggableModule.js»: логика портов для соединений

В листинге 2.4 показано, как с помощью хуков «useDrag» и «useDrop» реализуется интерактивность портов. Для выходных портов создается источник перетаскивания («useOutputPortDrag»), который передает идентификатор порта-источника. Для входных портов создается цель для «падения» («useInputPortDrop»), которая проверяет возможность соединения (порт не занят, не происходит самосоединение) и, в случае успеха, вызывает функцию «onConnect» для фиксации нового соединения в общем состоянии приложения. Визуальное отображение

портов и их доступности для соединения (например, изменение цвета) также управляется на основе состояний, получаемых от этих хуков.

2.2.4 Панель свойств выбранного модуля («PropertiesScreen.js»)

Компонент «PropertiesScreen.js» предоставляет пользователю интерфейс для просмотра и изменения параметров активного, то есть выбранного на основном рабочем поле, гидравлического модуля. Когда пользователь выбирает какой-либо модуль на схеме путем клика по нему, информация об этом модуле (объект «selectedModule») передается в «PropertiesScreen», и панель обновляется, отображая актуальные свойства данного элемента. Это позволяет инженеру детально конфигурировать каждый компонент гидросистемы.

Свойства: Гидроблок (ГБФ)

(ID экземпляра: 1747144287821)

Тип: block (GRU)

Цвет модуля:



Гидравлические параметры

Перепад давления (ΔP_a ном.):

0,3 МПа

Номинальный расход ($Q_{ном.}$):

14,72 л/мин

Внутренние утечки ($Q_{ут}$):

0,015 л/мин

Тепловые параметры

Площадь пов. (тепло):

0,063 м²

Сохранить
свойства

Удалить
модуль

Сбросить по
умолчанию

Рисунок 2.5 – Свойства модуля

Основные функции и логика работы компонента «PropertiesScreen.js» включают:

– При выборе нового модуля его текущие параметры (хранящиеся в свойстве «properties» объекта модуля) загружаются в локальное состояние компонента «PropertiesScreen». Если для каких-то параметров значения отсутствуют (например, при первом добавлении модуля из каталога), используются значения по умолчанию. Эти значения по умолчанию определены в родительском компоненте «App.js» в функции «getDefaultProperties» и специфичны для каждого типа модуля. Панель также отображает имя выбранного модуля и его уникальный идентификатор экземпляра для наглядности.

– Пользователь может изменять значения отображаемых свойств. Для каждого свойства предусмотрен соответствующий элемент управления: числовые поля для ввода количественных параметров (например, рабочий объем насоса, диаметр поршня), текстовые поля, выпадающие списки для выбора из predetermined вариантов (например, режим работы двигателя), или специальный элемент для выбора цвета модуля. При изменении значения в любом из полей ввода немедленно срабатывает функция-обработчик «handlePropertyChange». Эта функция обновляет локальное состояние «properties» компонента, записывая новое значение. В процессе обновления происходит базовая валидация данных: числовые значения преобразуются в соответствующий тип, значения коэффициентов полезного действия (КПД) автоматически ограничиваются диапазоном от 0 до 1, а физические размеры (длины, диаметры) не могут быть отрицательными.

– После того как пользователь внес необходимые корректировки в свойства модуля, он может применить их, нажав кнопку «Сохранить свойства». Это действие вызывает функцию «handleSave», которая, в свою очередь, обращается к функции «updateModuleProperties», переданной из «App.js». В эту функцию передаются идентификатор измененного модуля и новый набор его свойств. Таким образом, изменения сохраняются в общем состоянии приложения, и модуль на схеме обновляется.

– Панель свойств также предоставляет функционал для удаления выбранного модуля со схемы. Кнопка «Удалить модуль» активирует вызов функции «onDeleteModule» (также из «App.js»). Перед фактическим удалением программа запрашивает у пользователя подтверждение. Важно отметить, что ключевые системные модули, такие как «Двигатель» (тип «engine_input») и «Гидробак» (тип «tank_output»), защищены от удаления, так как они являются обязательными для функционирования любой схемы. При удалении модуля также автоматически удаляются все гидравлические соединения, связанные с ним.

– Если пользователь хочет отменить внесенные изменения или вернуть параметры модуля к стандартным для его типа, он может использовать кнопку «Сбросить по умолчанию». Это действие вызывает функцию «getDefaultProperties» из «App.js» для текущего типа модуля и устанавливает полученные значения в локальное состояние панели свойств.

– Состав полей для редактирования свойств не является статичным, а динамически изменяется в зависимости от типа выбранного модуля. Например, для модуля «Насос» будут показаны поля для ввода рабочего объема, КПД, передаточного числа привода и т.д., в то время как для модуля «Трубопровод» актуальными будут поля для внутреннего диаметра, длины и абсолютной шероховатости. Эта логика реализована внутри компонента с помощью условного рендеринга (чаще всего через конструкцию «switch» по типу или идентификатору модуля) в функции, ответственной за отрисовку полей свойств («renderProperties»).

– В дополнение к редактируемым свойствам, панель может отображать и результаты расчетов, относящиеся к выбранному модулю, если они были получены от сервера. Например, для гидроцилиндра это могут быть расчетные давления в полостях, скорость штока, развиваемая мощность. Эта информация выводится в отдельном блоке и не подлежит редактированию пользователем, служа для анализа работы компонента в текущей конфигурации схемы.


```

import React, { useState, useEffect } from 'react';
// ... импорт стилей

const PropertiesScreen = ({ selectedModule, updateModuleProperties,
onDeleteModule, detailedResults, getDefaultProperties }) => {
  const [properties, setProperties] = useState({}); // Локальное
состояние для редактируемых свойств
  const [moduleName, setModuleName] = useState('');
  // ... (прочее состояние, например, для calculatedCylinderParams)

  // useEffect для обновления состояния компонента при выборе другого
модуля
  useEffect(() => {
    if (selectedModule) {
      // Загружаем свойства выбранного модуля или пустой объект, если
их нет
      setProperties(selectedModule.properties || {});
      setModuleName(selectedModule.name || '');
      // ... (логика для загрузки расчетных параметров, если они есть
для этого модуля)
    } else {
      // Сброс состояния, если ни один модуль не выбран
      setProperties({});
      setModuleName('');
      // ...
    }
  }, [selectedModule, detailedResults]); // Зависимости эффекта

  // Обработчик изменения значения в любом поле свойства
  const handlePropertyChange = (e) => {
    const { name, value, type, checked } = e.target; // Получаем
атрибуты измененного элемента
    let processedValue;

    if (type === 'checkbox') {
      processedValue = checked;
    } else if (type === 'number') {
      const numValue = parseFloat(value);
      // Пример валидации: КПД не может быть < 0 или > 1
      if (name === 'volumetricEff' || name === 'mechEff' || name ===
'volEff') {
        if (numValue < 0) processedValue = 0;
        else if (numValue > 1) processedValue = 1;
        else processedValue = numValue || 0; // Если значение не
число, ставим 0
      } else if (['pistonDiameter', 'rodDiameter', 'stroke', 'force',
'length'].includes(name)) {
        // Пример: неотрицательные значения для размеров и усилий
        processedValue = Math.max(0, numValue || 0);
      } else {
        processedValue = numValue || 0;
      }
    } else {
      processedValue = value; // Для текстовых полей и цвета

```

```

    }
    // Обновляем локальный стейт свойств
    setProperties(prevProps => ({
      ...prevProps,
      [name]: processedValue
    }));
  });

const handleSave = () => {
  if (selectedModule) {
    // Вызываем функцию обратного вызова для сохранения свойств в
    глобальном состоянии
    updateModuleProperties(selectedModule.instanceId, properties);
    alert(`Свойства для «${moduleName}» сохранены!`);
  }
};

// ... (реализация handleDelete и handleResetToDefaults) ...

// Вспомогательная функция для генерации стандартного поля ввода
const renderInputField = (label, name, unit = '', type = 'number',
step = 'any', props = {}, tooltipText = '') => (
  <div key={name} className="property-field" title={tooltipText ||
label}>
    <label htmlFor={`prop-${name}`}>{label}</label>
    <div className="input-wrapper">
      <input
        type={type}
        id={`prop-${name}`}
        name={name}
        // Используем ?? для установки значения по умолчанию, если
свойство отсутствует
        value={properties[name] ?? (type === 'number' ? 0 : (type
=== 'color' ? '#9e9e9e' : ''))}
        onChange={handlePropertyChange}
        step={step}
        {...props} // Позволяет передавать доп. атрибуты, например,
min, max
      />
      {unit && <span className="unit">{unit}</span>}
    </div>
  </div>
);

// ... (вспомогательные функции renderRadioGroup,
renderPropertyGroupTitle) ...

if (!selectedModule) {
  return <div className="properties-screen">Выберите модуль для
настройки</div>;
}

// Функция, определяющая, какие именно поля свойств нужно отобразить

```

```

// в зависимости от типа (`selectedModule.id` или
`selectedModule.type`) выбранного модуля
const renderProperties = () => {
  switch (selectedModule.id) {
    case 'engine_d245':
      return (
        <>
          {renderPropertyGroupTitle('Параметры двигателя')}
          {renderInputField('Обороты XX (Nxx)', 'idleRpm',
'об/мин', 'number', 'any', { min: 0 })}
          {renderInputField('Ном. обороты (Nном)', 'nominalRpm',
'об/мин', 'number', 'any', { min: 0 })}
          {renderInputField('Обороты макс. момента (Nmax)',
'maxTorqueRpm', 'об/мин', 'number', 'any', { min: 0 })}
          {/* ... (renderRadioGroup для selectedRpmMode) ... */}
        </>
      );
    case 'pump_gns_ap30':
    case 'pump_gru_nsh10':
      return (
        <>
          {renderPropertyGroupTitle('Основные параметры насоса')}
          {renderInputField('Рабочий объем (Vн)', 'workingVolume',
'см³/об', 'number', 'any', { min: 0 })}
          {renderInputField('Передачное число (i)',
'driveRatio', '', 'number', 'any', { min: 0 })}
          {renderInputField('Объемный КПД (ηv)', 'volumetricEff',
'', 'number', '0.01', { min: 0, max: 1 })}
          {renderInputField('Мех. КПД (ηм)', 'mechEff', '',
'number', '0.01', { min: 0, max: 1 })}
          {/* ... (другие поля для насоса, например,
sideSurfaceArea) ... */}
        </>
      );
    // ... (другие case для cylinder, distributor, filter, pipe и
т.д.) ...
    default:
      return <p>Специфические свойства для данного модуля не
определены.</p>;
  }
};

// ... (функция renderCalculatedParam для отображения расчетных
данных) ...

return (
  <div className="properties-screen">
    <h2>Свойства: {moduleName}</h2>
    <p
      className="module-info">(ID
      {selectedModule.instanceId})экземпляра:
      {selectedModule.id}</p>
    <p
      className="module-info">Тип:
      {selectedModule.type}
      {selectedModule.system ? `(${selectedModule.system.toUpperCase()})`
      : ''}</p>
  </div>
);

```

```

        {renderInputField('Цвет модуля', 'color', '', 'color',
undefined, {}, "Выберите цвет для отображения модуля на схеме")}
        <hr />
        <div className="specific-properties">
            {renderProperties()} {/* Вызов функции для динамического
рендеринга */}
        </div>

        {/* Блок отображения расчетных параметров (если есть) */}
        {selectedModule.type === 'cylinder' && calculatedCylinderParams
&& (
            <>
                <hr />
                <div className="calculated-parameters-section">
                    {renderPropertyGroupTitle('Расчетные
цилиндра' )}
                    {/* ... (вызовы renderCalculatedParam для каждого
параметра) ... */}
                </div>
            </>
        )}
        {/* ... */}
        <hr />
        <div className="properties-actions">
            <button
                onClick={handleSave}
disabled={!selectedModule}>Сохранить свойства</button>
            {selectedModule && (selectedModule.type !== 'engine_input' &&
selectedModule.type !== 'tank_output') && (
                <button
                    onClick={handleDelete}
                    className="delete-
button">Удалить модуль</button>
            )}
            {selectedModule && getDefaultProperties && (
                <button
                    onClick={handleResetToDefaults}
className="reset-button">Сбросить по умолчанию</button>
            )}
        </div>
    </div>
);
};
export default PropertiesScreen;

```

Листинг 2.5 – Обработка изменения свойств и динамический рендеринг полей

В Листинге 2.5 продемонстрирована логика компонента «PropertiesScreen.js». Хук «useEffect» отслеживает изменения в «selectedModule» и при выборе нового модуля загружает его свойства в локальное состояние «properties», что позволяет пользователю их редактировать. Функция «handlePropertyChange» обрабатывает ввод пользователя, производит базовую валидацию (например, для КПД или неотрицательных значений) и обновляет локальное состояние. Сохранение происходит через вызов «updateModuleProperties».

Особенно важна функция «renderProperties». Она использует конструкцию «switch» по идентификатору («selectedModule.id») или типу («selectedModule.type»)

выбранного модуля для динамического формирования набора полей ввода. Это позволяет отображать только те параметры, которые релевантны для данного конкретного гидравлического компонента, делая интерфейс чище и понятнее. Вспомогательная функция «renderInputField» унифицирует создание элементов ввода, что упрощает код и обеспечивает единообразный вид панели свойств.

2.2.5 Модальное окно загрузки схем («LoadSchemeModal.js»)

Для обеспечения возможности повторного использования ранее созданных и сохраненных гидравлических схем в приложении реализовано модальное окно загрузки. Компонент «LoadSchemeModal.js» отвечает за отображение этого окна и взаимодействие с пользователем в процессе выбора и загрузки схемы.

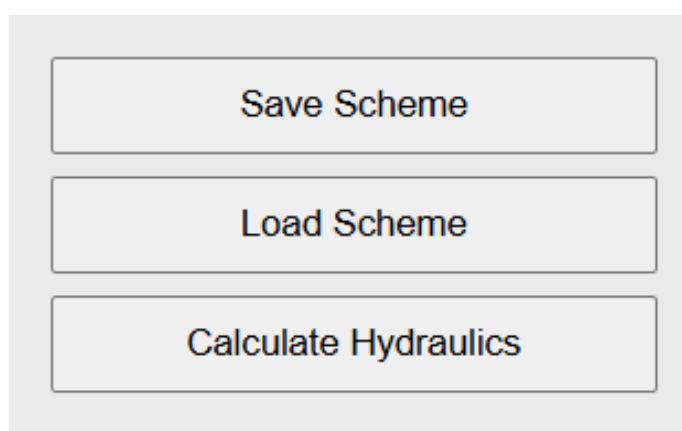


Рисунок 2.6 – Панель работы со базой данных схем

Функционал и реализация компонента:

- Когда пользователь инициирует загрузку схемы (например, нажатием соответствующей кнопки в главном интерфейсе), состояние «isLoadModalOpen» в родительском компоненте «App.js» устанавливается в «true», что делает модальное окно видимым. При открытии окна компонент отображает список ранее сохраненных пользователем гидравлических схем. Этот список получается из пропса «schemes», который, в свою очередь, загружается из базы данных при монтировании «App.js» или принудительно перед открытием модального окна (через вызов «fetchSavedSchemes»). Каждая схема в списке представлена своим именем и датой последнего обновления, что помогает пользователю ориентироваться.

- Пользователь может выбрать интересующую его схему из отображаемого списка простым кликом по ее названию.

- Поскольку загрузка новой схемы приведет к замене текущей, возможно, несохраненной схемы на рабочем поле, перед выполнением операции выводится стандартное диалоговое окно браузера («window.confirm») с предупреждением и запросом подтверждения. Это стандартная мера предосторожности для предотвращения случайной потери данных.

- Если пользователь подтверждает свое намерение, вызывается функция «onLoad» (которая является ссылкой на функцию «handleLoadScheme» из компонента «App.js»). В эту функцию передается уникальный идентификатор

(«_id») выбранной схемы. Функция «handleLoadScheme» отправляет запрос на сервер для получения полных данных указанной схемы из базы данных. После успешного получения данных, состояние массивов «mainScreenModules» и «connections» в «App.js» обновляется, и на рабочем поле отображается загруженная схема.

– Окно загрузки можно закрыть несколькими способами: нажатием на кнопку «Отмена» или кликом мыши на полупрозрачном фоне (overlay) за пределами основного контента модального окна.

– Во время выполнения асинхронных операций, таких как запрос списка схем с сервера или загрузка конкретной схемы, пользователю отображается текстовый индикатор (например, «Загрузка списка схем...» или «Загрузка...»), а кнопки выбора схем и закрытия окна могут быть временно деактивированы («disabled={isLoading}»). Это обеспечивает обратную связь и предотвращает нежелательные действия во время обработки запросов.

```
// src/components/LoadSchemeModal.js
import React from 'react';
import '../styles/LoadSchemeModal.css'; // Подключение стилей для
модального окна

const LoadSchemeModal = ({ isOpen, onClose, schemes, onLoad, isLoading
}) => {
  // Если модальное окно не должно быть открыто (isOpen === false),
  компонент ничего не рендерит
  if (!isOpen) {
    return null;
  }

  // Обработчик для закрытия окна при клике на фон (overlay)
  const handleOverlayClick = (e) => {
    if (e.target === e.currentTarget) { // Убеждаемся, что клик
    был именно по overlay
      onClose(); // Вызываем функцию закрытия, переданную из
    родительского компонента
    }
  };

  // Обработчик клика по элементу списка (названию схемы)
  const handleSchemeClick = (schemeId) => {
    // Запрашиваем подтверждение у пользователя перед заменой
    текущей схемы
    if (window.confirm("Loading a scheme will replace the current
    diagram. Are you sure?")) {
      onLoad(schemeId); // Вызываем функцию загрузки,
    переданную из родительского компонента
    }
  };

  return (
    // Полупрозрачный фон, покрывающий все окно браузера
    <div className="modal-overlay" onClick={handleOverlayClick}>
      {/* Контейнер для содержимого модального окна */}

```

```

        <div className="modal-content">
            <h2>Загрузить схему</h2>

            {/* Условный рендеринг в зависимости от состояния
загрузки и наличия схем */}
            {isLoading && schemes.length === 0 && <p>Загрузка
списка схем...</p>}
            {!isLoading && schemes.length === 0 && <p>Сохраненные
схемы не найдены.</p>}

            {/* Отображение списка схем, если они есть */}
            {schemes.length > 0 && (
                <ul className="scheme-list">
                    {schemes.map(scheme => ( // Итерация по
массиву схем
                        <li
                            key={scheme._id} // Уникальный ключ
для каждого элемента React-списка
                                onClick={() => !isLoading &&
handleSchemeClick(scheme._id)} // Обработчик клика, неактивен во
время загрузки
                                    className={isLoading ? 'disabled' :
''} // CSS-класс для стилизации неактивного элемента
                                >
                                    {scheme.name} {/* Отображение имени
схемы */}
                                </li>
                            )}}
                    </ul>
                )}

            {/* Кнопка для закрытия модального окна */}
            <button
                                onClick={onClose}
disabled={isLoading}>Отмена</button>
            </div>
        </div>
    );
};

export default LoadSchemeModal;

```

Листинг 2.6 – Отображение списка схем и обработка выбора

Компонент «LoadSchemeModal.js», как показано в Листинге 2.6, реализует стандартное модальное окно. Его отображение контролируется состоянием «isOpen». Список доступных для загрузки схем передается через пропс «schemes».

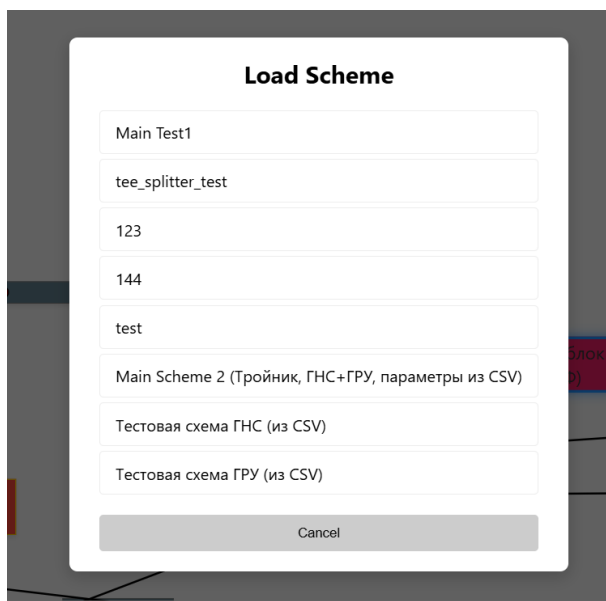


Рисунок 2.7 – Панель загрузки схем

При выборе пользователем одной из схем и подтверждении действия, инициируется процесс загрузки через вызов функции «onLoad». Вся логика по взаимодействию с сервером для получения списка схем и загрузки конкретной схемы инкапсулирована в родительском компоненте «App.js».

2.2.6 Панель отображения результатов

После того как серверная часть выполнила все необходимые гидравлические и тепловые расчеты на основе построенного пользователем схемы, полученные результаты должны быть представлены в наглядной и информативной форме. За эту задачу отвечает компонент «ResultsDisplay.js». Он структурирует и отображает как общие сводные данные по каждой отдельной гидравлической системе (например, для системы гидронавесного оборудования или системы рулевого управления, если они питаются от разных насосов), так и детализированную информацию по каждой ветке внутри этих систем и по отдельным гидравлическим компонентам, включая каждый участок трубопровода.

Результаты выводятся в логической последовательности, отражающей структуру гидропривода. Сначала идут общие данные по системе, затем информация по каждой ее ветви, и внутри веток – параметры конкретных компонентов, таких как гидроцилиндры, и участков трубопроводов.

Для каждой независимой системы, обычно сгруппированной вокруг одного насоса, отображаются:

- Фактическая подача рабочей жидкости насосом и развиваемое им давление на выходе.
- Мощность, потребляемая насосом от приводного двигателя.
- Интегральные показатели эффективности: общий (полный) КПД данной системы, а также ее глобальные гидравлический и объемный КПД, рассчитанные от выхода насоса до исполнительных гидродвигателей.

Если система имеет разветвления (например, после тройника или многосекционного распределителя), для каждой такой ветки, ведущей к потребителю (например, гидроцилиндру) или к гидробаку (в случае сливной линии), выводится:

- Требуемое давление на входе данной ветки для обеспечения работы ее компонентов.

- Коэффициенты полезного действия (гидравлический, механический, объемный и общий) для «чистой» ветки, то есть рассчитанные от точки входа в эту ветку до ее конечного элемента.

- В случае если конечным элементом ветки является гидроцилиндр, отображаются его ключевые расчетные параметры: давления в поршневой и штоковой полостях, фактическая скорость движения штока, полезная мощность, развиваемая на штоке, и действительный расход жидкости, поступающий в цилиндр.

Для каждого индивидуального участка трубопровода, как в напорных, так и в сливных линиях (включая общие участки и участки внутри отдельных веток), предоставляется следующая информация:

- Идентификатор или имя трубы.
- Геометрические параметры: внутренний диаметр и длина.
- Гидродинамические параметры: расчетная средняя скорость потока жидкости, число Рейнольдса (Re), коэффициент гидравлического трения (λ).
- Расчетные потери давления на трение и местные потери для данного участка трубы, если они были вычислены. Скорость потока может быть визуально выделена (например, цветом), если она выходит за пределы рекомендуемых значений для данного типа гидрелинии (всасывающая, напорная или сливная), что помогает пользователю быстро оценить корректность выбора диаметров.

В отдельном блоке представляется информация об общем тепловом балансе всей спроектированной гидросистемы (учитывая все активные насосы и их подсистемы). Эта секция включает:

- Общее количество тепла, выделяемого в гидросистеме в результате потерь мощности.
- Суммарная площадь поверхности теплообмена всего гидрооборудования (насосов, распределителей, цилиндров, фильтров и т.д., исключая гидробак).
- Эффективная площадь теплоотдающей поверхности используемого гидробака.
- Расчетная установившаяся температура рабочей жидкости в гидробаке при заданных условиях окружающей среды.
- Целевая максимальная температура рабочей жидкости (например, 70°C).
- Если расчетная температура превышает целевую, выводится информация о необходимой дополнительной площади теплообмена или требуемой эффективной площади бака.

- Итоговое заключение о состоянии теплового баланса: достаточно ли существующего охлаждения (за счет бака и естественной конвекции от оборудования) или требуется установка дополнительного теплообменника (радиатора).

```
import React from 'react';
```

```

import    '../styles/ResultsDisplay.css';    //    Стили    для    панели
результатов

const ResultsDisplay = ({ resultsData, allModules }) => {
    // Проверка на наличие данных для отображения
    if (!resultsData || !resultsData.details ||
Object.keys(resultsData.details).length === 0) {
        return null; // или <p>Нет данных для отображения.</p>;
    }

    // Вспомогательная функция для форматирования числовых значений
и добавления единиц измерения
    const formatValue = (value, unit = '', precision = 2) => {
        if (value === null || value === undefined) {
            return `Н/Д ${unit}`; // Если данных нет
        }
        if (typeof value === 'number') {
            if (Number.isNaN(value)) {
                return `Ошибка ${unit}`; // Если результат расчета -
NaN
            }
            // Для очень малых чисел используется экспоненциальная
форма
            if (Math.abs(value) < 1e-9 && value !== 0) {
                return `${value.toExponential(precision)} ${unit}`;
            }
            return `${value.toFixed(precision)} ${unit}`; //
Стандартное форматирование
        }
        return `${value} ${unit}`; // Для нечисловых значений
(например, текстовых заключений)
    };

    // Функция для определения стиля отображения скорости потока
    // (например, подсветка синим при низкой скорости, красным при
высокой)
    const getSpeedStyle = (speedMs, lineTypeForStyle) => {
        let style = {};
        const lowerLineType = lineTypeForStyle.toLowerCase();
        // Примерные пороговые значения для стилизации
        if (lowerLineType.includes('напорная')) {
            if (speedMs < 3.0) style.color = 'blue'; // Ниже
рекомендуемой
            else if (speedMs > 5.0) style.color = 'red'; // Выше
рекомендуемой
        } else if (lowerLineType.includes('сливная')) {
            if (speedMs < 2.0) style.color = 'blue';
            else if (speedMs > 3.0) style.color = 'red';
        }
        return style;
    };

    // Функция для рендеринга таблицы с детализацией по трубопроводам

```

```

const renderPipeDetails = (pipes, lineDescription,
lineTypeForStyle) => {
    if (!pipes || pipes.length === 0) {
        return <p>Нет данных по индивидуальным трубам для
{lineDescription}</p>;
    }
    return (
        <div className="pipe-details-section">
            <strong>Трубы {lineDescription}</strong>
            <table>
                <thead>
                    <tr>
                        <th>Название/ID</th><th>D (мм)</th><th>L
(м)</th>
                        <th>Скорость
(м/с)</th><th>Re</th><th> $\lambda$ </th>
                    </tr>
                </thead>
                <tbody>
                    {pipes.map(pipe => ( // Итерация по массиву
труб
                        <tr key={pipe.instanceId}>
                            <td>{pipe.name}
({String(pipe.instanceId).slice(-4)})</td>
                            <td>{formatValue(pipe.diameterM *
1000, '', 1)}</td>
                            <td>{formatValue(pipe.lengthM, '',
2)}</td>
                            <td
style={getSpeedStyle(pipe.velocityMs,
lineTypeForStyle)}>{formatValue(pipe.velocityMs, '', 3)}</td>
                            <td>{formatValue(pipe.reynolds, '',
0)}</td>
                            <td>{formatValue(pipe.lambda, '',
4)}</td>
                        </tr>
                    ))}
                </tbody>
            </table>
        </div>
    );
};

// Функция для рендеринга блока с информацией о тепловом балансе
const renderThermalBalance = (thermalData) => {
    if (!thermalData) {
        // ... (обработка отсутствия данных)
        return <p>Данные теплового баланса отсутствуют.</p>;
    }
    // ... (извлечение и форматирование данных из thermalData)
    return (
        <div className="thermal-balance-section system-card">
            <h4>Тепловой Баланс Системы (Общий)</h4>
            <ul>

```

```

        <li>Общее тепловыделение (Q1):
{formatValue(thermalData.totalHeatGeneratedKw, 'кВт', 3)}</li>
        <li>Эффективная площадь пов. гидробака (Fбпр):
{formatValue(thermalData.currentEffectiveTankAreaM2, 'м²', 2)}</li>
        <li style={{ fontWeight: 'bold' }}>Расчетная
установившаяся температура (tж расч):
{formatValue(thermalData.calculatedSteadyStateTempC, '°C', 1)}</li>
        { /* ... (другие параметры и заключение) ... */ }
    </ul>
    <p><strong>Заключение по тепловому расчету:</strong>
{thermalData.conclusion}</p>
    </div>
    );
};

return (
    <div className="results-display-container">
        <h3>Детализация Расчетов Системы:</h3>
        { /* Итерация по каждой системе (например, ГНС, ГРУ) */ }
        {resultsData.details &&
Object.entries(resultsData.details).map(([systemKey, systemData]) =>
(
            <div key={systemKey} className="system-card">
                <h4>Система: {systemKey.replace(/_/g, ' ')}</h4>
                <div className="system-summary">
                    { /* ... (отображение общих данных по насосу
и КПД системы) ... */ }
                </div>

                <h5>Ветки данной системы:</h5>
                {systemData.branches &&
Object.entries(systemData.branches).map(([branchKey, branchData]) =>
(
                    <div key={branchKey} className="branch-
card">
                        <h6>Ветка: {branchKey.replace(/_/g, '
')}</h6>
                        { /* ... (отображение данных по ветке) ...
*/ }

                        {branchData.cylinderCalculatedParams &&
(
                            <div className="cylinder-details">
                                { /* ... (отображение расчетных
параметров цилиндра) ... */ }
                            </div>
                        ) }
                        { /* Вызов функции для рендеринга таблиц
с трубами */ }

                        {branchData.pressureLinePipeDetails &&
renderPipeDetails(branchData.pressureLinePipeDetails, "напорной
линии ветки", "напорная")}

                        { /* ... (renderPipeDetails для сливных
линий ветки) ... */ }

```

```

        </div>
    ))}
    { /* ... (обработка случая отсутствия веток) ...
*/}
    { /* ... (отображение общих сливных линий после
коллекторов) ... */}
    </div>
    ))}
    { /* Отображение блока теплового баланса */}
    {resultsData.thermalBalance
renderThermalBalance(resultsData.thermalBalance)}
    &&
    </div>
    );
};

export default ResultsDisplay;

```

Листинг 2.7 – Структура отображения результатов расчета

Компонент «ResultsDisplay», как показано в сокращенном Листинге 2.7, предназначен для структурированного вывода результатов гидравлических и тепловых расчетов. Он принимает на вход объект «resultsData», содержащий детализацию по каждой гидросистеме и ее ветвям. Для наглядности и удобства анализа данные форматируются с помощью вспомогательной функции «formatValue».

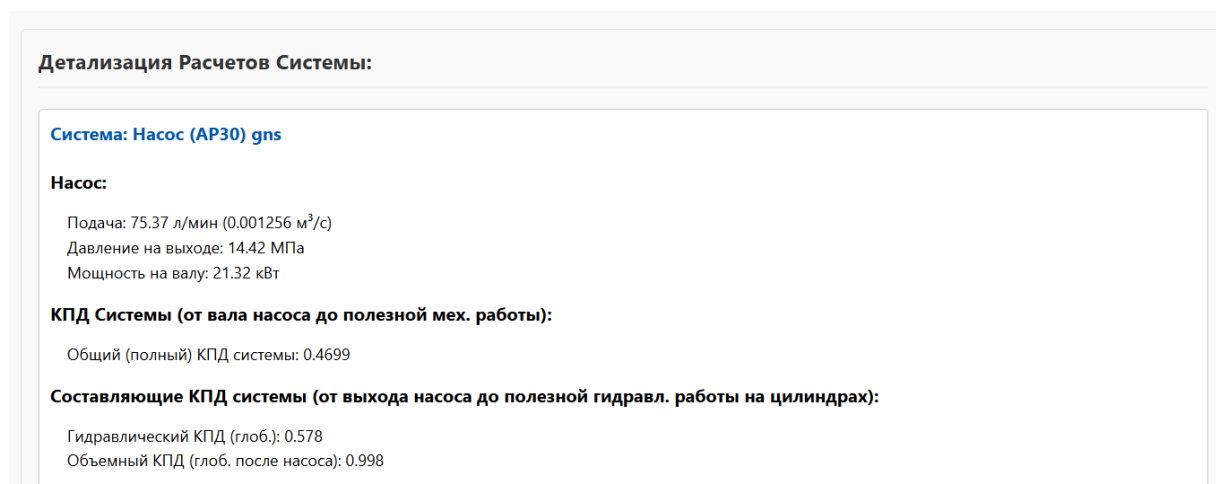


Рисунок 2.8 – Детализация расчётов

Информация по трубопроводам выводится в табличном виде функцией «renderPipeDetails», где также может применяться условное форматирование (например, подсветка скорости потока). Отдельная функция «renderThermalBalance» отвечает за отображение всех параметров, связанных с тепловым режимом системы. Такая компонентная структура позволяет гибко управлять отображением сложного набора расчетных данных.

2.3 Реализация серверной логики (Backend)

Серверная часть программного комплекса является ядром системы, ответственным за выполнение всех сложных вычислений, связанных с анализом гидравлических схем, а также за управление данными, включая сохранение и загрузку пользовательских проектов. Она обрабатывает запросы от клиентской части, производит расчеты и возвращает результаты для отображения.

2.3.1 Обзор используемых технологий (Node.js, Express)

В качестве технологической основы для серверной части была выбрана платформа Node.js. Это среда выполнения JavaScript, построенная на движке V8 от Google Chrome, которая позволяет использовать JavaScript для написания серверного кода. Ее асинхронная, событийно-ориентированная архитектура хорошо подходит для обработки множества одновременных запросов, что важно для веб-приложений.

Для упрощения разработки веб-сервера и управления маршрутизацией HTTP-запросов используется популярный фреймворк Express.js. Express предоставляет минималистичный и гибкий набор инструментов для создания веб-приложений и API, включая обработку HTTP-методов (GET, POST и т.д.), управление сессиями и куки, а также интеграцию с различным промежуточным программным обеспечением (middleware) для таких задач, как парсинг тел запросов или обработка ошибок.

Для взаимодействия с базой данных MongoDB, где хранятся гидравлические схемы, применяется библиотека Mongoose. Mongoose представляет собой инструмент объектно-документного моделирования (ODM) для Node.js, который предоставляет удобный API для определения схем данных, выполнения запросов и управления документами в базе данных.

2.3.2 API для взаимодействия с клиентской частью

Сервер предоставляет набор четко определенных конечных точек (API-эндпоинтов), через которые клиентское приложение может отправлять запросы и получать ответы.

2.3.2.1 API для расчета гидравлической системы

Этот эндпоинт является основным для выполнения расчетной части проекта. Он ожидает POST-запрос от клиента, тело которого содержит JSON-объект с полной информацией о гидравлической схеме. Эта информация включает массив объектов, описывающих все модули на схеме (их типы, свойства, заданные пользователем, и координаты), а также массив объектов, описывающих соединения между портами этих модулей.

Получив эти данные, серверная логика инициирует сложный процесс анализа и расчета:

- Сначала производится анализ топологии предоставленной схемы. Это включает идентификацию всех насосных контуров, определение направления

потоков жидкости, трассировку путей от насосов к гидродвигателям и далее к гидробаку, а также выявление параллельных веток, образующихся после разветвителей (тройников) или многосекционных распределителей.

- Затем для каждого участка трубопровода в каждой идентифицированной ветке рассчитываются основные гидродинамические параметры: скорость течения рабочей жидкости и число Рейнольдса.

- На основе числа Рейнольдса и характеристик трубы (диаметр, шероховатость) определяется коэффициент гидравлического трения, после чего вычисляются потери давления на трение по длине трубопровода.

- Рассчитываются потери давления в местных сопротивлениях (изгибы, сужения, расширения, арматура) и непосредственно в элементах гидроаппаратуры (клапаны, распределители, фильтры) на основе их паспортных данных и фактического расхода.

- Далее определяются фактические давления в различных точках гидросистемы, включая рабочие полости гидродвигателей (цилиндров).

- Вычисляются развиваемые гидродвигателями усилия или моменты, их скорости или частоты вращения, а также полезные и потребляемые мощности.

- Оценивается общий коэффициент полезного действия (КПД) гидропривода в целом и его отдельных подсистем.

- Производится тепловой расчет для определения установившейся температуры рабочей жидкости и оценки необходимости применения дополнительных средств охлаждения.

По завершении всех вычислений сервер формирует структурированный JSON-ответ. Этот ответ содержит как общие результаты по каждой системе (например, КПД, общие потери), так и детализированные данные по каждому элементу и участку трубопровода (скорости, давления, потери), включая результаты теплового расчета. Этот ответ отправляется обратно на клиентскую часть для последующего отображения пользователю.

2.3.2.2 API для сохранения и загрузки схем («/api/schemes»)

Для обеспечения возможности длительного хранения пользовательских наборок и их повторного использования, сервер предоставляет API для управления схемами в базе данных:

- POST «/api/schemes»: Этот эндпоинт используется для сохранения новой или обновленной гидравлической схемы. Клиент отправляет имя схемы и ее полные данные (массив модулей и массив соединений). Сервер создает новый документ в коллекции схем в базе данных MongoDB или обновляет существующий.

- GET «/api/schemes»: По этому запросу сервер возвращает клиенту список всех сохраненных гидравлических схем. Обычно для оптимизации передаются только метаданные, такие как имя схемы, ее уникальный идентификатор и дата последнего изменения. Эта информация используется на клиенте для отображения списка доступных для загрузки схем.

- GET «/api/schemes/:id»: Этот эндпоинт позволяет загрузить полную информацию о конкретной схеме по ее уникальному идентификатору («id»). Клиент

использует этот запрос, когда пользователь выбирает схему для загрузки из списка. Сервер находит соответствующий документ в базе данных и отправляет его клиенту.

2.3.3 Алгоритм обработки гидравлической схемы

Центральным элементом серверной логики является алгоритм, который детально анализирует структуру (топологию) гидравлической схемы, предоставленной клиентом, и на основе этого анализа выполняет все необходимые расчеты.

2.3.3.1 Поиск и трассировка путей в гидросистеме

Для корректного расчета потерь давления и распределения потоков необходимо точно определить пути, по которым рабочая жидкость движется через различные компоненты схемы. Алгоритм выполняет следующие шаги:

- Первым делом находятся все насосы, присутствующие на схеме. Каждый насос может формировать отдельную гидравлическую систему или подсистему.
- Для каждого насоса производится трассировка: всасывающей линии от гидробака (модуль типа «tank_output») до входа насоса, напорной линии: от выхода насоса к исполнительным механизмам или разветвителям.
- Поиск следующего элемента. Для построения пути используется функция, аналогичная «findNextModuleInstance», представленная в серверном коде. Эта функция на основе идентификатора текущего модуля (или его конкретного выходного порта) и общего списка соединений определяет, какой модуль является следующим в гидравлической цепи.

```
function findNextModuleInstance(currentModuleIdOrPortId,
allConnections, allModules) {
    // currentModuleIdOrPortId - может быть ID самого модуля или
    ID его конкретного выходного порта (например, для тройника)
    const sourceIdentifier = String(currentModuleIdOrPortId);

    // Ищем соединение, которое исходит из данного порта/модуля
    const outgoingConnection = allConnections.find(connection =>
String(connection.sourceId) === sourceIdentifier);

    if (!outgoingConnection) {
        return null; // Если исходящих соединений нет, путь здесь
заканчивается
    }

    // Извлекаем ID целевого модуля из ID целевого порта
    // (отбрасываем суффиксы типа "_in0", "_in1" для
коллекторов)
    let targetModuleInstanceId =
String(outgoingConnection.targetId);
    if (targetModuleInstanceId.includes('_in')) {
        targetModuleInstanceId =
targetModuleInstanceId.split('_in')[0];
```



```

    }

    // Находим и возвращаем объект целевого модуля по его
instanceId
    return allModules.find(module => String(module.instanceId)
=== targetModuleInstanceId);
    }

```

Листинг 2.8 – Фрагмент серверной функции для поиска следующего модуля в цепи

В Листинге 2.8 показан принцип работы функции «findNextModuleInstance». Она принимает идентификатор текущего узла (или его порта) и массивы всех соединений и всех модулей на схеме. Функция ищет соединение, которое начинается от указанного источника. Если такое соединение найдено, извлекается идентификатор целевого порта, по которому затем определяется идентификатор целевого модуля. После этого из общего списка модулей выбирается и возвращается объект этого целевого модуля. Если исходящего соединения нет, функция возвращает «null», сигнализируя об окончании данного пути.

Для более сложных сценариев, например, для определения полного пути от гидродвигателя обратно к баку (сливная линия), используется функция «findPathModules». Этот алгоритм, как правило, реализует один из методов обхода графа (например, поиск в глубину) для нахождения всех компонентов, составляющих указанный путь, избегая при этом заикливания за счет отслеживания уже посещенных узлов.

2.3.3.2 Рекурсивное обнаружение ветвей

Схемы гидроприводов часто содержат элементы, разделяющие поток жидкости на несколько параллельных ветвей, например, тройники или многосекционные гидрораспределители. Для корректного расчета таких систем необходимо точно идентифицировать каждую ветвь и определить расход жидкости в ней. Функция «discoverBranchesRecursive», присутствующая в серверном коде, предназначена для решения этой задачи.

Принцип ее работы следующий:

- Трассировка начинается от узла, следующего за насосом или предыдущим элементом, разделяющим поток.

- Если в процессе трассировки встречается модуль типа «splitter» (тройник), алгоритм понимает, что здесь происходит разделение потока. Функция рекурсивно вызывает саму себя для каждого из выходных портов этого тройника. При этом входящий в тройник поток жидкости («currentFlowM3s») распределяется между новыми ветвями (в простейшем случае – поровну, если иное не задано логикой работы тройника или регуляторами потока на ветвях). Путь, пройденный до тройника, сохраняется и передается в рекурсивные вызовы как «начальные компоненты» для новых веток.

– Если трассировка достигает конечного элемента ветви, такого как гидроцилиндр («cylinder») или гидробак («tank_output»), текущий прослеженный путь регистрируется как отдельная, завершенная гидравлическая ветвь. Сохраняется информация о всех модулях, составляющих эту ветвь, и о потоке жидкости в ней.

– Если текущий узел не является ни разветвителем, ни конечным потребителем, трассировка просто продолжается к следующему соединенному с ним модулю.

– Для предотвращения бесконечных циклов в схемах, которые могут содержать замкнутые контуры (хотя в типичных гидроприводах это редкость для основных силовых линий), используется механизм отслеживания уже посещенных узлов в рамках текущего пути рекурсивной трассировки («visitedOnThisTraceSet»). Если алгоритм пытается повторно войти в уже посещенный на данном пути узел, рекурсия для этой траектории прерывается.

```
function discoverBranchesRecursive(
  currentNode, // Текущий модуль, с которого начинается или
                // продолжается анализ ветки
  currentFlowM3s, // Расход жидкости, входящий в данный узел или
                // ветку
  pathFromLastSplitterOrPump, // Список модулей на пути от
                // предыдущего разветвителя или насоса до currentNode
  initialPressureComponentsForThisBranch, // Список модулей от
                // насоса до первого разветвителя на данном общем пути
  pumpSystemType, // Идентификатор системы насоса (например, 'gns'
                // или 'gru')
  connections, // Общий список всех соединений на схеме
  modules, // Общий список всех модулей на схеме
  tankId, // InstanceId гидробака
  branchesArrayOutput, // Массив, куда будут добавляться найденные
                // ветки
  visitedOnThisTraceSet, // Set для отслеживания посещенных узлов
                // на текущем пути рекурсии
  originalPumpFlowM3s, // Исходный полный поток от насоса данной
                // системы
  currentBranchNamePrefix // Префикс для именования ветки (для
                // отладки и логов)
) {
  const currentNodeIdStr = String(currentNode.instanceId);
  const currentTraceKey =
    `${currentNodeIdStr}_${pathFromLastSplitterOrPump.map(m =>
      m.instanceId).join(',')}_${initialPressureComponentsForThisBranch.map(m => m.instanceId).join(',')}`;

  // Проверка на заикливание для текущей траектории
  if (visitedOnThisTraceSet.has(currentTraceKey)) {
    console.warn(`[discoverBranchesRecursive] Обнаружен цикл
или повторный обход узла ${currentNode.name}. Остановка.`);
    return;
  }
  visitedOnThisTraceSet.add(currentTraceKey);

  if (currentNode.type === 'splitter') {
```

```

        const splitter = currentNode;
        // ... (логика поиска выходных соединений из splitterOutputs
у тройника) ...
        const splitterOutputs = connections.filter(c =>
String(c.sourceId).startsWith(String(splitter.instanceId) +
"_out"));

        if (splitterOutputs.length > 0) {
            // Пример: делим поток поровну (в реальности может быть
сложнее)
            const flowPerSplitterOutput = currentFlowM3s /
splitterOutputs.length;
            splitterOutputs.forEach((splitConn, index) => {
                // ... (определяем nextNodeAfterSplitter по ID
целевого порта splitConn.targetId)
                const nextNodeAfterSplitter = modules.find(m =>
String(m.instanceId) === String(splitConn.targetId).split('_')[0]);

                if (nextNodeAfterSplitter) {
                    // Рекурсивный вызов для каждой новой ветки после
тройника
                    discoverBranchesRecursive(
                        nextNodeAfterSplitter,
                        flowPerSplitterOutput,
                        [], // Новый путь начинается от этого
тройника
                        // Путь до текущего тройника становится
начальным для новых веток
                        [...initialPressureComponentsForThisBranch,
...pathFromLastSplitterOrPump, splitter],
                        // ... (остальные параметры передаются
дальше)
                        pumpSystemType, connections, modules,
                        tankId, branchesArrayOutput,
                        new Set(visitedOnThisTraceSet), /* Новый Set
для каждого пути */
                        originalPumpFlowM3s,
                        `${currentBranchNamePrefix}_split${index}_to_${nextNodeAfterSplitter
.name.replace(/[(\)\s]/g, '')}`
                    );
                }
            });
        } else {
            // Если тройник не имеет выходов, это тупик
            branchesArrayOutput.push({ /* ... информация о тупиковой
ветке, заканчивающейся на тройнике ... */ });
        }
    } else if (currentNode.type === 'cylinder' || currentNodeIdStr
=== String(tankId)) {
        // Достигли конечного элемента ветки (гидроцилиндр или
гидробак)

```

```

        const          entryNodeForThisDefinedBranch          =
pathFromLastSplitterOrPump.length > 0 ? pathFromLastSplitterOrPump[0]
: currentNode;
        // ... (логика определения originatingSplitterPort, если
ветка началась от тройника) ...
        branchesArrayOutput.push({
            branchName: currentBranchNamePrefix,
            entryNodeInstanceId:
entryNodeForThisDefinedBranch.instanceId,
            actualEntryNodeForPathCalc: currentNode, // Конечный узел
этой ветки
            flowM3s: currentFlowM3s,
            initialComponentsOnPressurePath:
initialPressureComponentsForThisBranch,
            // ... (другая информация о ветке)
        });
    } else {
        // Промежуточный узел (не тройник и не конечный потребитель)
        // ... (ищем следующий узел nextNode через
findNextModuleInstance) ...
        const          nextNode          =
findNextModuleInstance(currentNode.instanceId,          connections,
modules);
        if (nextNode) {
            // Продолжаем трассировку по текущей ветке
            discoverBranchesRecursive(
                nextNode,
                currentFlowM3s, // Поток пока не меняется (если нет
утечек на currentNode)
                [...pathFromLastSplitterOrPump, currentNode], //
Добавляем текущий узел к пути
                initialPressureComponentsForThisBranch,
                // ... (остальные параметры)
                pumpSystemType, connections, modules, tankId,
branchesArrayOutput,
                new Set(visitedOnThisTraceSet), originalPumpFlowM3s,
currentBranchNamePrefix
            );
        } else {
            // Если следующего узла нет - это тупиковая ветка
            branchesArrayOutput.push({ /* ... информация о тупиковой
ветке, заканчивающейся на currentNode ... */ });
        }
    }
}

```

Листинг 2.9 – Структура функции «discoverBranchesRecursive» для идентификации ветвей

В Листинге 2.9 представлена концептуальная структура функции «discoverBranchesRecursive». Она осуществляет рекурсивный обход графа гидравлической схемы, начиная от выхода насоса или разветвителя. Если текущий узел является разветвителем («splitter»), функция порождает новые рекурсивные

вызовы для каждого из его выходных потоков. Если достигается конечный потребитель (например, «cylinder») или гидробак, текущий путь идентифицируется как отдельная ветвь, и информация о ней (включая участвующие компоненты и величину потока) сохраняется. Такой подход позволяет корректно анализировать системы со сложной топологией, включая параллельные контуры.

2.3.4 Реализация гидравлических расчетов на сервере

После того как топология гидравлической схемы проанализирована и определены все пути потоков и отдельные ветви, сервер приступает к выполнению гидравлических расчетов. Эти расчеты включают определение скоростей, потерь давления, рабочих давлений на гидродвигателях и общих характеристик системы.

2.3.4.1 Расчет скорости потока и числа Рейнольдса

Для каждого участка трубопровода в каждой идентифицированной ветви, зная расход рабочей жидкости Q (определенный на этапе анализа топологии и деления потоков) и внутренний диаметр трубы d , вычисляется средняя скорость потока v . Эта скорость определяется по известной формуле неразрывности потока: $v = (4 \text{ умножить на } Q) \text{ разделить на } (\pi \text{ умножить на } d \text{ в квадрате})$.

Далее, используя рассчитанную скорость, диаметр трубы и кинематическую вязкость рабочей жидкости ν (которая зависит от типа масла и его температуры), определяется число Рейнольдса (Re) по формуле: $Re = (v \text{ умножить на } d) \text{ разделить на } \nu$. Число Рейнольдса является безразмерным критерием, характеризующим режим течения жидкости: ламинарный, если Re меньше критического значения (обычно около 2300), или турбулентный, если Re больше этого значения. Режим течения важен для последующего выбора формулы для коэффициента гидравлического трения. В коде серверной части эти вычисления инкапсулированы в функциях, подобных «calculateVelocity» и «calculateReynoldsNumber».

2.3.4.2 Расчет коэффициента гидравлического трения (λ)

Коэффициент гидравлического трения λ необходим для расчета потерь давления на трение по длине трубопровода. Его значение зависит от режима течения жидкости (числа Рейнольдса) и относительной шероховатости внутренней поверхности трубы.

- Для ламинарного режима течения ($Re < 2300$), коэффициент λ обычно рассчитывается по формуле Пуазейля: $\lambda = A / Re$, где A – константа (часто 64 или 75).

- Для турбулентного режима течения ($Re > 2300$), используются более сложные зависимости. Для гидравлически гладких труб при умеренных числах Re может применяться формула Блазиуса. При больших числах Re и для шероховатых труб применяются формулы, учитывающие относительную шероховатость, например, формула Альтшуля или Кольбрука-Уайта. Серверная функция «calculateLambda» реализует эту логику выбора соответствующей формулы.

2.3.4.3 Расчет потерь давления на трение и местных потерь

Потери давления в гидросистеме складываются из потерь на трение по длине трубопроводов и потерь в местных гидравлических сопротивлениях.

– Потери давления на трение ($\Delta P_{\text{тр}}$) в прямолинейном участке трубопровода определяются по формуле Дарси-Вейсбаха: $\Delta P_{\text{тр}} = \lambda$ умножить на (L/d) умножить на $(\rho$ умножить на v в квадрате / 2), где L – длина трубы, d – ее внутренний диаметр, ρ – плотность жидкости, v – скорость потока. Эта логика реализована в серверной функции «calculateFrictionLoss».

– Местные потери давления ($\Delta P_{\text{м}}$) возникают при изменении формы или направления потока в различных элементах, таких как изгибы, тройники, внезапные сужения или расширения канала, а также во входных и выходных участках. Они рассчитываются по общей формуле: $\Delta P_{\text{м}} = b$ умножить на $\Sigma \xi$ умножить на $(\rho$ умножить на v в квадрате / 2), где $\Sigma \xi$ – сумма коэффициентов местных сопротивлений на рассматриваемом участке, а « b » – поправочный коэффициент, учитывающий влияние режима течения (для ламинарного режима « b » может быть больше единицы и зависит от числа Re, для турбулентного « b » обычно принимается равным единице). Значения коэффициентов ξ берутся из справочной литературы или задаются в свойствах соответствующих модулей. Эта часть расчета выполняется функцией «calculateLocalLoss».

– Потери давления непосредственно в гидроаппаратуре (такой как распределители, клапаны, фильтры) обычно определяются на основе их паспортных данных. В каталогах указывается номинальный перепад давления при номинальном расходе. Если фактический расход через аппарат отличается от номинального, перепад давления пересчитывается, как правило, по квадратичной зависимости от отношения фактического расхода к номинальному.

```
// Функция для расчета потерь в одной линии (напорной или сливной)
const lineCalculations = (
  lineDescription,          // Строковое описание линии для логирования
  modulesOnPath,            // Массив модулей, составляющих этот
  участок линии
  flowM3s,                  // Расход жидкости через линию, м³/с
  defaultOverallZetaForLine, // Общий коэффициент местных
  сопротивлений для всей линии (если нет данных по отдельным трубам)
  fluidProperties,          // Объект со свойствами рабочей жидкости
  (плотность, кинематическая вязкость)
  DEFAULT_PIPE_DIAMETER,    // Диаметр трубы по умолчанию, если не
  задан в свойствах модуля
  DEFAULT_PIPE_LENGTH,      // Длина трубы по умолчанию
  DEFAULT_PIPE_ROUGHNESS    // Шероховатость трубы по умолчанию
) => {
  // ... (вывод в лог информации о начале расчета для данной линии)
  ...
  let totalFrictionLossPa = 0; // Аккумулятор потерь на трение
  let totalLocalLossPa = 0;    // Аккумулятор местных потерь
  const pipeDetailsArray = []; // Массив для сбора детализированной
  информации по каждой трубе на участке

  // Обрабатываем только модули типа «pipe» на данном пути
```

```

    const pipesOnThisPath = modulesOnPath.filter(m => m.type ===
'pipe');
    if (pipesOnThisPath.length > 0) {
        pipesOnThisPath.forEach((pipe) => {
            const pipeProps = pipe.properties || {};
            // Получаем параметры трубы из ее свойств или используем
значения по умолчанию
            const diameter = pipeProps.diameter ||
DEFAULT_PIPE_DIAMETER;
            const length = pipeProps.length || DEFAULT_PIPE_LENGTH;
            const roughness = pipeProps.roughness ||
DEFAULT_PIPE_ROUGHNESS;
            const localZetaPipe = pipeProps.localResistanceCoeff ||
0; // Коэф. местных потерь, заданный для этого участка трубы

            const velocity = calculateVelocity(flowM3s, diameter); //
Вспомогательная функция
            // ... (можно добавить расчет средней скорости по всем
трубам, если это необходимо) ...

            const reynolds = calculateReynoldsNumber(velocity,
diameter, fluidProperties.kinematicViscosityM2s); // Вспомогательная
функция
            const lambda = calculateLambda(reynolds, diameter,
roughness); // Вспомогательная функция

            const frictionLoss = calculateFrictionLoss(lambda,
length, diameter, fluidProperties.density, velocity); //
Вспомогательная функция
            const localLossInPipe =
calculateLocalLoss(localZetaPipe, fluidProperties.density, velocity,
reynolds); // Вспомогательная функция

            totalFrictionLossPa += frictionLoss; // Суммируем потери
на трение
            totalLocalLossPa += localLossInPipe; // Суммируем
местные потери внутри трубы

            // Сохраняем детализированную информацию по этой трубе
pipeDetailsArray.push({
                instanceId: pipe.instanceId,
                name: pipe.name || 'Трубопровод',
                diameterM: parseFloat(diameter.toFixed(4)), //
Округление для отображения
                lengthM: parseFloat(length.toFixed(2)),
                velocityMs: parseFloat(velocity.toFixed(3)),
                reynolds: parseFloat(reynolds.toFixed(0)),
                lambda: parseFloat(lambda.toFixed(4)),
                frictionLossPa: parseFloat(frictionLoss.toFixed(0)),
                localLossInPipePa:
parseFloat(localLossInPipe.toFixed(0)),
            });
        });
    }

```

```

        // Если для всей линии задан общий коэффициент местных
сопротивлений (и не было индивидуальных по трубам)
        // или для учета потерь в не-трубопроводных элементах линии
(например, изгибы без явного модуля "колесо")
        if (defaultOverallZetaForLine > 0 && pipesOnThisPath.length ===
0) { // Условие немного упрощено для примера
            const averageVelocityOverall = calculateVelocity(flowM3s,
DEFAULT_PIPE_DIAMETER); // Пример расчета по умолч. диаметру
            const averageReynoldsOverall =
calculateReynoldsNumber(averageVelocityOverall,
DEFAULT_PIPE_DIAMETER, fluidProperties.kinematicViscosityM2s);
            totalLocalLossPa +=
calculateLocalLoss(defaultOverallZetaForLine,
fluidProperties.density, averageVelocityOverall,
averageReynoldsOverall);
        }

        return {
            totalFrictionLossPa: totalFrictionLossPa,
            totalLocalLossPa: totalLocalLossPa,
            pipeDetails: pipeDetailsArray,
            // ... (могут возвращаться и другие агрегированные данные,
например, средняя скорость)
        };
    };
};

```

Листинг 2.10 – Фрагмент серверной функции для расчета потерь в линии

В Листинге 2.10 представлена концепция функции «lineCalculations». Эта функция принимает на вход описание участка гидравлической линии (массив модулей, составляющих этот участок), расход жидкости через него и свойства рабочей жидкости. Она итерирует по всем модулям типа «pipe» на данном участке. Для каждого трубопровода, используя его геометрические параметры (диаметр, длина, шероховатость) и свойства жидкости, вычисляются скорость потока, число Рейнольдса, коэффициент гидравлического трения λ , и, наконец, потери давления на трение и локальные потери (если для данного участка трубы задан коэффициент местного сопротивления). Эти потери суммируются для получения общих потерь на трение и общих местных потерь для всего рассматриваемого участка линии. Детализированные расчетные параметры для каждого трубопровода (скорость, Re, λ и т.д.) сохраняются для последующего отображения пользователю.

2.3.4.5 Расчет давлений в полостях гидродвигателей и на выходе насоса

Зная суммарные потери давления на всех участках напорной гидролинии (от насоса до гидродвигателя) и сливной гидролинии (от гидродвигателя до бака), можно определить фактические рабочие давления в полостях гидродвигателей. Например, для гидроцилиндра, работающего на выдвижение штока:

- Давление в его поршневой (нагнетательной) полости будет равно давлению на выходе из насоса за вычетом всех потерь давления в напорной линии, включая потери в распределителе, клапанах и трубопроводах, ведущих к этой полости.

- Давление в его штоковой (сливной) полости будет равно сумме всех потерь давления в сливной линии, от штоковой полости до гидробака, включая потери в фильтре (если он установлен на сливе), распределителе и трубопроводах.

Далее, используя уравнение равновесия сил, действующих на поршень гидроцилиндра (с учетом полезной нагрузки R и силы трения T), можно определить, какое давление должно быть создано в нагнетательной полости для выполнения заданной работы. В серверном коде эта логика содержится в функции «calculateCylinderPressure».

Давление, которое фактически должен развивать насос (P_n), определяется как максимальное из давлений, требуемых на входе каждой из параллельных веток гидросистемы (или непосредственно на гидродвигателе, если ветка одна), плюс все потери давления на общем участке пути от насоса до точки разветвления или до этого гидродвигателя. Серверная логика анализирует все активные ветви, рассчитывает для каждой из них необходимое давление на входе и выбирает наибольшее значение. Это и будет расчетное давление на выходе насоса.

2.3.4.6 Расчет мощностей и КПД

После определения всех фактических давлений, расходов и скоростей для каждого элемента и системы в целом, производится расчет мощностных характеристик и коэффициентов полезного действия:

- Полезная механическая мощность, развиваемая каждым гидродвигателем (цилиндром или гидромотором), рассчитывается на основе фактического усилия/момента и скорости/частоты вращения.

- Мощность, потребляемая каждым насосом от приводного двигателя, определяется на основе его фактической подачи, развиваемого давления на выходе и его полного (или механического и объемного) КПД.

- Общий КПД каждой независимой гидравлической системы (например, системы ГНС или ГРУ) рассчитывается как отношение суммарной полезной механической мощности, развиваемой всеми гидродвигателями этой системы, к суммарной мощности, потребляемой насосом (или насосами), питающим эту систему.

- Дополнительно могут быть рассчитаны и представлены пользователю отдельные составляющие КПД, такие как гидравлический КПД системы (учитывающий только гидравлические потери в линиях и аппаратуре) и объемный КПД системы (учитывающий суммарные утечки рабочей жидкости после насоса).

2.3.5 Реализация теплового расчета на сервере

Тепловой расчет является важной частью анализа гидропривода, так как перегрев рабочей жидкости может серьезно повлиять на его работоспособность и долговечность. Серверная часть выполняет следующие шаги для оценки теплового баланса:

– Расчет общего количества тепла, выделяемого в гидросистеме ($Q_{\text{ген}}$). Это тепло эквивалентно суммарным потерям мощности во всех активных гидросистемах. Потери мощности в каждой системе вычисляются как разница между мощностью, потребляемой ее насосом (или насосами), и суммарной полезной механической мощностью, развиваемой ее гидродвигателями $Q_{\text{ген}} = \Sigma(N_{\text{насоспотр}} - N_{\text{гдвполезн}})$.

– Определение суммарной площади поверхности теплоотдачи всего гидрооборудования ($F_{\text{об}}$), за исключением гидробака. Эта площадь вычисляется как сумма значений параметра «sideSurfaceArea» (площадь боковой поверхности для теплообмена), заданных в свойствах каждого модуля (насосов, распределителей, цилиндров, фильтров, трубопроводов и т.д.).

– Расчет эффективной площади теплоотдающей поверхности гидробака ($F_{\text{б.эфф}}$). Эта площадь зависит от его геометрических размеров (длины, ширины, высоты уровня жидкости в баке) и обычно принимается как часть его полной наружной поверхности (например, 90% от общей площади поверхности бака, контактирующей с воздухом).

– Вычисление установившейся температуры рабочей жидкости ($t_{\text{ж.уст}}$). Эта температура определяется из условия теплового баланса, когда количество тепла, генерируемого в системе, равно количеству тепла, отдаваемого в окружающую среду через поверхности оборудования и бака: $t_{\text{ж.уст}} = t_{\text{окр.ср}} + Q_{\text{ген}} / (K \cdot (F_{\text{об}} + F_{\text{б.эфф}}))$, где $t_{\text{окр.ср}}$ – температура окружающей среды (задается как входной параметр), K – общий усредненный коэффициент теплопередачи от поверхностей гидросистемы к воздуху ($\text{Вт}/(\text{м}^2\text{°C})$), который зависит от условий обдува.

– Сравнение расчетной установившейся температуры с максимально допустимой температурой для используемой рабочей жидкости (обычно 70-75 °C). Если $t_{\text{ж.уст}}$ превышает допустимое значение, формируется заключение о том, что текущей площади теплоотдачи недостаточно. В этом случае может быть дополнительно рассчитана требуемая общая площадь теплообмена или требуемая эффективная площадь поверхности гидробака для поддержания температуры в заданных пределах. Если и этого недостаточно, делается вывод о необходимости применения специального теплообменника (радиатора).

2.4 Работа с базой данных

Для обеспечения возможности сохранения и последующего использования созданных пользователями гидравлических схем, программный комплекс включает интеграцию с системой управления базами данных (СУБД). В данном проекте в качестве СУБД используется MongoDB.

2.4.1 Схема данных для сохранения гидравлических схем

Для структурированного хранения информации о каждой гидравлической схеме в базе данных MongoDB определяется специальная схема (модель) документа. В серверном коде эта схема определена с использованием библиотеки Mongoose и называется «SchemeSchema».

«name»: Строковое поле, предназначенное для хранения имени, присвоенного пользователем данной гидравлической схеме. Это поле является обязательным и помогает идентифицировать схему в списке сохраненных проектов.

«data»: Вложенный объект, который содержит всю информацию о структуре и параметрах гидравлической схемы. Этот объект, в свою очередь, включает два ключевых массива:

– «modules»: Массив объектов, где каждый объект описывает один гидравлический модуль, размещенный на рабочем поле. Для каждого модуля сохраняются его тип («id», «type», «system»), имя («name»), уникальный идентификатор экземпляра («instanceId»), текущие координаты на холсте («position»), а также объект со всеми его настраиваемыми свойствами («properties»).

– «connections»: Массив объектов, описывающих все гидравлические соединения между портами модулей. Каждый объект соединения содержит его уникальный идентификатор («id»), идентификатор порта-источника («sourceId») и идентификатор порта-приемника («targetId»).

«timestamps»: Mongoose автоматически добавляет поля «createdAt» и «updatedAt», которые фиксируют время создания и последнего обновления документа схемы соответственно. Это полезно для сортировки схем по дате и отслеживания изменений.

```
const mongoose = require('mongoose');

const SchemeSchema = new mongoose.Schema({
  name: { type: String, required: true }, // Имя схемы, обязательное поле
  data: { // Объект, содержащий данные о схеме
    modules: { type: Array, required: true }, // Массив объектов модулей
    connections: { type: Array, required: true }, // Массив объектов соединений
  },
}, { timestamps: true }); // Автоматическое добавление полей createdAt и updatedAt

// Создание модели 'Scheme' на основе определенной схемы
const Scheme = mongoose.model('Scheme', SchemeSchema);

// ... (далее экспорт модели, если она в отдельном файле)
// module.exports = Scheme;
```

Листинг 2.11 – Определение схемы для хранения гидравлических схем

В Листинге 2.11 показано, как с помощью Mongoose определяется структура документа для хранения гидравлической схемы. Поле «name» является обязательным. Поле «data» содержит два обязательных массива: «modules» и «connections», которые в точности соответствуют структуре данных, используемой на клиентской части для представления схемы. Опция «timestamps: true» автоматически добавляет временные метки.

2.4.2 Операции CRUD для схем

Серверная часть реализует основные операции для управления сохраненными схемами, которые часто объединяют под акронимом CRUD (Create, Read, Update, Delete).

Когда пользователь решает сохранить текущую гидравлическую схему, клиентская часть отправляет POST-запрос на эндпоинт «/api/schemes». Тело запроса содержит имя схемы и ее данные (модули и соединения). Серверная логика создает новый экземпляр модели «Scheme» с полученными данными и сохраняет его в базе данных MongoDB.

```
app.post('/api/schemes', async (req, res) => {
  try {
    const { name, data } = req.body;
    // Проверка на наличие необходимых данных
    if (!name || !data || !Array.isArray(data.modules) ||
    !Array.isArray(data.connections)) {
      return res.status(400).json({ error: 'Неверный формат
данных. Требуется: name, data: { modules: [], connections: [] }' });
    }
    const newScheme = new Scheme({ name, data }); // Создание
нового документа Mongoose
    await newScheme.save(); // Асинхронное сохранение в базу
данных
    res.status(201).json(newScheme); // Ответ клиенту с созданным
объектом и статусом 201 (Created)
  } catch (error) {
    // ... (обработка ошибок сохранения)
    res.status(500).json({ error: 'Не удалось сохранить схему',
details: error.message });
  }
});
```

Листинг 2.12 – Обработчик POST-запроса для сохранения новой схемы

Для получения списка всех сохраненных схем клиент отправляет GET-запрос на «/api/schemes». Сервер извлекает из базы данных все документы из коллекции схем (или только их метаданные, такие как имя и дата, для оптимизации) и отправляет их клиенту. В коде это запрос, сортирующий схемы по дате обновления.

Для загрузки конкретной схемы клиент отправляет GET-запрос на «/api/schemes/:id», где «:id» – это уникальный идентификатор схемы в базе данных. Сервер находит соответствующий документ и возвращает его полные данные клиенту.

```
app.get('/api/schemes', async (req, res) => {
  try {
    // Запрос к базе данных для получения всех схем,
    // выбираем только поля _id, name, updatedAt и сортируем по
дате обновления в убывающем порядке
    const schemes = await Scheme.find().select('_id name
updatedAt').sort('-updatedAt');
```

```

        res.status(200).json(schemes); // Отправляем список схем
клиенту
    } catch (error) {
        // ... (обработка ошибок чтения)
        res.status(500).json({ error: 'Не удалось получить список
схем', details: error.message });
    }
});

```

Листинг 2.13 – Обработчик GET-запроса для получения списка схем

Таким образом, взаимодействие с базой данных MongoDB через Mongoose позволяет программному комплексу эффективно управлять пользовательскими гидравлическими схемами, обеспечивая их сохранение, загрузку и организацию.

2.5 Выводы по разделу

Во второй главе было подробно рассмотрено устройство и принципы работы разработанного программного комплекса, предназначенного для проектирования и расчета объемных гидроприводов. Была описана его клиент-серверная архитектура, обеспечивающая разделение задач между пользовательским интерфейсом и вычислительной логикой.

Клиентская часть, реализованная с использованием библиотеки React и React DnD, предоставляет пользователю интуитивно понятный графический интерфейс. Ключевыми элементами являются:

- Каталог гидравлических модулей, из которого компоненты перетаскиваются на рабочее поле.
- Основное рабочее поле («MainScreen») с поддержкой панорамирования и масштабирования, где происходит визуальное построение гидравлических схем и соединение модулей.
- Панель свойств («PropertiesScreen»), позволяющая детально настраивать параметры каждого выбранного элемента схемы, а также отображающая результаты расчетов для конкретного модуля.
- Функционал сохранения и загрузки схем, реализованный через модальное окно («LoadSchemeModal») и взаимодействие с серверной частью.
- Панель отображения результатов («ResultsDisplay»), наглядно представляющая итоги гидравлических и тепловых расчетов.

Серверная часть, построенная на Node.js с использованием фреймворка Express и библиотеки Mongoose для работы с MongoDB, выполняет следующие функции:

- Обрабатывает запросы от клиента через определенные API-эндпоинты, в частности, для расчета гидравлических характеристик схемы и для операций с сохраненными схемами (создание, чтение).
- Реализует сложные алгоритмы анализа топологии гидравлической схемы, включая трассировку путей потока и рекурсивное обнаружение параллельных ветвей.

- Производит все необходимые гидравлические вычисления (скорости, числа Рейнольдса, коэффициенты трения, потери давления, рабочие давления, мощности, КПД) и тепловой расчет системы.

- Взаимодействует с базой данных MongoDB для персистентного хранения и извлечения данных о гидравлических схемах, используя определенную Mongoose-схему.

Разработанный программный комплекс автоматизирует значительную часть рутинных вычислений, связанных с проектированием гидроприводов, позволяет быстро оценивать различные варианты схемных решений и обеспечивает удобное хранение проектов. Это делает его полезным инструментом для инженеров-гидравликов и студентов, изучающих соответствующие дисциплины.

3 Проектный расчет объемного гидропривода

В данной главе демонстрируются возможности разработанного программного комплекса на примере расчета нескольких гидравлических схем. Целью является иллюстрация процесса ввода исходных данных пользователем, а также представление и анализ результатов, получаемых с помощью программы. Для обеспечения краткости изложения, основное внимание будет сосредоточено на ключевых выходных параметрах и их интерпретации.

3.1 Общие исходные данные для примеров расчета

Если в описании конкретного примера не указано иное, для всех расчетов принимаются следующие общие параметры:

Рабочая жидкость: Масло гидравлическое HLP32.

– Плотность (ρ): 868 кг/м³.

– Кинематическая вязкость (ν) при стандартной расчетной температуре: 33,2 сСт (что эквивалентно $33,2 \cdot 10^{-6}$ м²/с).

Параметры окружающей среды:

– Температура окружающего воздуха ($t_{в\max}$): 30 °С.

– Общий усредненный коэффициент теплопередачи от поверхностей гидрооборудования к воздуху (K): 15 Вт/(м²·°С).

Приводной двигатель (для всех насосов): Тип Д245. Конкретные значения оборотов (холостого хода, номинальные, при максимальном крутящем моменте) задаются пользователем в свойствах модуля «Двигатель» для каждого отдельного примера расчета.

Гидробак: если в примере не указаны другие размеры, используются геометрические параметры по умолчанию: длина $L = 0,695$ м, ширина $S = 0,36$ м, высота $H = 0,445$ м.

3.2 Пример 1: Расчет гидронавесной системы (ГНС)

3.2.1 Описание схемы и исходных данных

В данном примере рассматривается гидравлическая схема навесной системы. Она включает двигатель, насос типа AP30, набор последовательно соединенных трубопроводов, гидрораспределитель модели RGE100, блок питания BPG и один силовой гидроцилиндр Ц63, используемый для операций задненавесного устройства (ЗНУ). Слив рабочей жидкости из гидроцилиндра осуществляется через сливной фильтр FRC12 и далее по трубопроводам обратно в общий гидробак. Эта конфигурация схемы соответствует данным, представленным в файле «Тестовая схема ГНС (из CSV)». Для проведения расчета выбран режим работы приводного двигателя, соответствующий оборотам максимального крутящего момента.

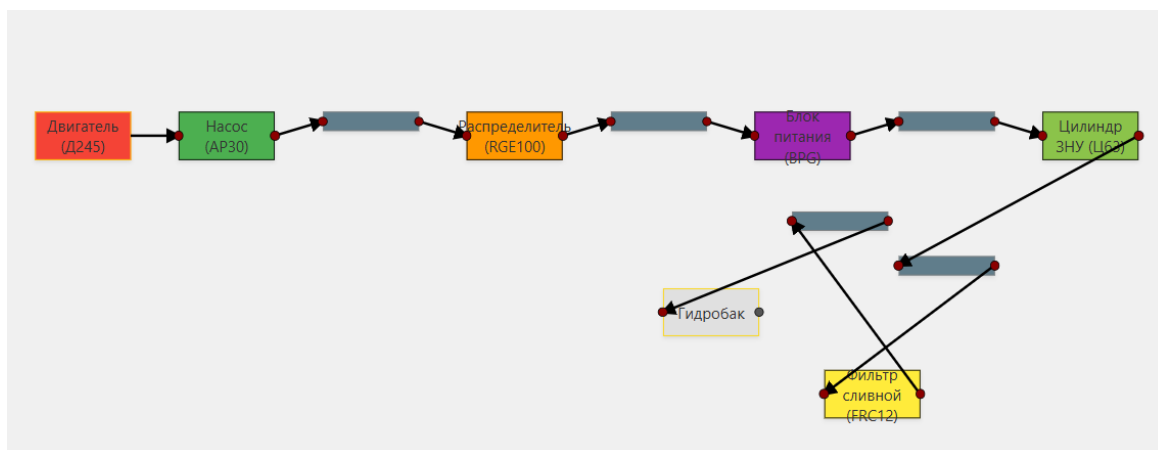


Рисунок 3.1 – Схема ГНС

Все ключевые параметры элементов гидросистемы (рабочие объемы, диаметры, длины, коэффициенты сопротивлений, КПД и т.д.) задаются пользователем в интерфейсе программы, в панели свойств для каждого модуля, в соответствии с данными из файла схемы или используются значения по умолчанию, логика определения которых была описана в Главе 2.

3.2.2 Результаты расчета и их анализ

После ввода всех необходимых параметров и запуска процедуры расчета, программный комплекс предоставляет следующие основные выходные данные.

Параметры насоса (AP30) для системы ГНС:

- Подача: приблизительно 54,81 л/мин.
- Давление на выходе насоса: приблизительно 15,40 МПа.
- Мощность, потребляемая насосом на валу: приблизительно 16,55 кВт.

Коэффициент полезного действия системы ГНС (рассчитанный от вала насоса до полезной механической работы на штоке гидроцилиндра): приблизительно 0,7053 (или 70,53%).

Параметры гидроцилиндра ЗНУ (Ц63):

- Давление в поршневой (нагнетательной) полости: приблизительно 13,51 МПа.
- Скорость движения штока: приблизительно 0,2924 м/с.
- Полезная механическая мощность, развиваемая на штоке: приблизительно 11,68 кВт.

Результаты теплового баланса для данной системы:

- Общее количество тепла, выделяемого системой: приблизительно 4,88 кВт.
- Расчетная установившаяся температура рабочей жидкости в гидробаке: приблизительно 187,7 °С.

Закключение по тепловому режиму: требуется установка дополнительного теплообменника (радиатора), так как расчетная установившаяся температура рабочей жидкости (187,7 °С) значительно превышает максимально допустимую температуру (обычно около 70-75 °С для масел HLP).

Анализ полученных результатов показывает, что основные рабочие параметры гидроцилиндра (давление в полостях, косвенно определяющее усилие, и скорость

штока) соответствуют ожидаемым для выполнения рабочих операций. Однако система характеризуется значительным тепловыделением, что указывает на необходимость принятия мер по улучшению охлаждения рабочей жидкости. Общий КПД системы, составляющий примерно 70.5%, можно считать приемлемым для такого типа гидроприводов. Детальная информация по каждому участку трубопровода (включая скорости потока, числа Рейнольдса, коэффициенты трения и потери давления), также предоставляемая программой, позволяет инженеру оценить правильность выбора диаметров труб и выявить участки с наибольшими гидравлическими потерями.

3.3 Пример 2: Расчет гидросистемы рулевого управления (ГРУ)

3.3.1 Описание схемы и исходных данных

В этом примере анализируется отдельная, независимая гидросистема, предназначенная для рулевого управления. Схема включает приводной двигатель, насос типа НШ10, напорный трубопровод, гидроблок модели ГБФ (который может включать распределительные и клапанные функции), силовой гидроцилиндр Ц70 (используемый в механизме поворота) и сливной трубопровод, ведущий в общий гидробак. Данная конфигурация соответствует файлу «Тестовая схема ГРУ (из CSV)». Расчет производится для режима работы приводного двигателя на оборотах максимального крутящего момента.

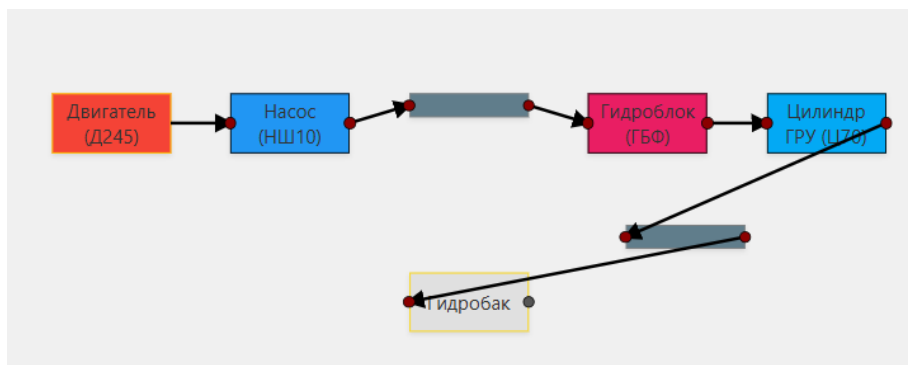


Рисунок 3.2 – Схема ГРУ

3.3.2 Результаты расчета и их анализ

Программный комплекс предоставляет следующие основные расчетные параметры для системы ГРУ:

Параметры насоса (НШ10) для системы ГРУ:

- Подача: приблизительно 14,72 л/мин.
- Давление на выходе насоса: приблизительно 3,67 МПа.
- Мощность, потребляемая насосом на валу: приблизительно 1,06 кВт.

Коэффициент полезного действия системы ГРУ: приблизительно 0,7370 (или 73,7%).

Параметры гидроцилиндра ГРУ (Ц70):

- Давление в поршневой (нагнетательной) полости: приблизительно 3,25 МПа.

- Скорость движения штока: приблизительно 0,0637 м/с.
- Полезная механическая мощность, развиваемая на штоке: приблизительно 0,78 кВт.

Результаты теплового баланса (при условии, что эта система работает изолированно, но использует общий гидробак и учитывается теплоотдача от оборудования из примера 1, если оно неактивно, или только от своих компонентов):

- Общее количество тепла, выделяемого только системой ГРУ: приблизительно 0,28 кВт.

- Расчетная установившаяся температура рабочей жидкости (при работе только ГРУ, но с учетом общей площади теплоотдачи бака и простаивающего оборудования из примера 1): приблизительно 43,3 °С.

- Заключение по тепловому режиму: При работе только системы рулевого управления тепловой режим находится в пределах нормы, перегрева не ожидается.

Система рулевого управления, как и ожидалось, характеризуется значительно более низкими рабочими давлениями и потребляемыми мощностями по сравнению с гидронавесной системой. Общий КПД составляет около 73.7%, что является хорошим показателем. Тепловыделение от данной системы само по себе невелико и при ее изолированной работе (или в комбинации с неработающей ГНС) не должно приводить к перегреву рабочей жидкости в общем гидробаке.

3.4 Пример 3: Расчет комбинированной гидросистемы

3.4.1 Описание схемы и исходных данных

В данном примере анализируется более сложная гидравлическая конфигурация, где один насос (AP30, относящийся к системе «gns») через тройник (разветвитель потока) одновременно питает два различных потребителя:

- Гидроцилиндр ЗНУ, аналогичный тому, что использовался в Примере 1.
- Контур, ведущий к гидроцилиндру ГРУ, аналогичному тому, что использовался в Примере 2, но в данном случае он запитан от насоса системы «gns». Слив рабочей жидкости от обоих гидроцилиндров осуществляется через общий коллектор (смеситель потоков) и далее через общую сливную линию в гидробак. Эта схема соответствует конфигурации, описанной как «Тестовая схема 3». Для данного расчета выбран режим работы приводного двигателя на номинальных оборотах.

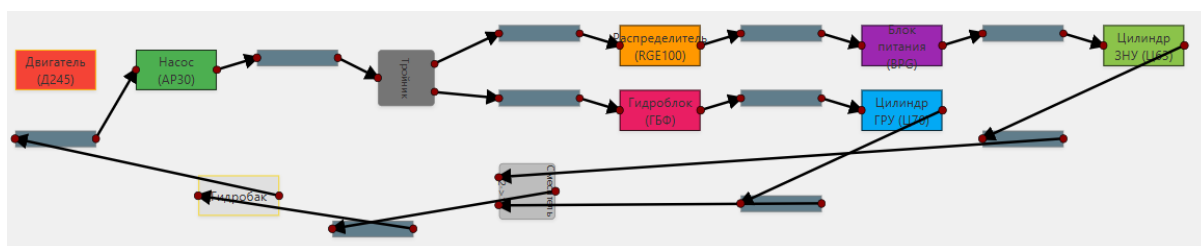


Рисунок 3.3 – Схема комбинированной гидросистемы

3.4.2 Результаты расчета и их анализ

Ключевые расчетные параметры для этой комбинированной системы:

Параметры насоса (AP30) системы «gns»:

- Общая подача насоса: приблизительно 75,37 л/мин (эта подача затем делится тройником между двумя ветками).

- Давление на выходе насоса: приблизительно 14,42 МПа. Это давление определяется веткой с наибольшим сопротивлением, то есть веткой гидроцилиндра ЗНУ.

- Мощность, потребляемая насосом на валу: приблизительно 21,32 кВт.

Коэффициент полезного действия общей системы «gns» (от вала насоса до суммарной полезной работы на обоих цилиндрах): приблизительно 0,4699 (или 47,0%).

Параметры ветки, ведущей к гидроцилиндру ЗНУ:

- Давление в поршневой полости: приблизительно 13,48 МПа.

- Скорость движения штока: приблизительно 0,2008 м/с (при потоке в эту ветку около 37,57 л/мин, что составляет примерно половину от общей подачи насоса).

- Полезная механическая мощность на штоке: приблизительно 8,02 кВт.

Параметры ветки, ведущей к гидроцилиндру ГРУ:

- Давление в поршневой полости: приблизительно 3,25 МПа (это давление определяется требуемым усилием на штоке цилиндра ГРУ, а не давлением насоса напрямую, так как поток дросселируется до этого значения из-за более высокого давления в параллельной ветке ЗНУ).

- Скорость движения штока: приблизительно 0,1631 м/с (при потоке в эту ветку около 37,67 л/мин).

- Полезная механическая мощность на штоке: приблизительно 2,00 кВт.

Результаты общего теплового баланса для комбинированной системы:

- Общее количество тепла, выделяемого системой: приблизительно 11,30 кВт.

- Расчетная установившаяся температура рабочей жидкости в гидробаке: приблизительно 420,9 °С.

- Заключение по тепловому режиму: требуется установка очень эффективного теплообменника (радиатора). Столь высокий расчетный перегрев указывает на существенные проблемы в эффективности данной конфигурации при одновременной работе обоих потребителей.

Анализ результатов для комбинированной схемы показывает, что при параллельном питании нескольких потребителей от одного нерегулируемого насоса через простое деление потока (например, с помощью тройника, который в простейшем случае делит поток поровну), общее давление, развиваемое насосом, определяется веткой, требующей наибольшего давления. В менее нагруженной ветке (в данном случае, ветка гидроцилиндра ГРУ) избыточное давление будет дросселироваться (теряться) на элементах этой ветки или на самом распределителе, что приводит к значительным потерям энергии. Это наглядно демонстрируется очень низким общим КПД системы (около 47%) и чрезвычайно высоким тепловыделением, приводящим к недопустимо высокой расчетной температуре рабочей жидкости. Такая схема требует либо пересмотра параметров (например, использования насоса с регулируемой подачей или отдельных насосов для каждого потребителя), либо применения более сложных схем деления потока с учетом нагрузки на каждой ветке, либо значительного улучшения системы охлаждения.

3.5 Выводы по разделу

В третьей главе была продемонстрирована работа разработанного программного комплекса на практических примерах расчета трех различных гидравлических систем: отдельной гидронавесной системы (ГНС), отдельной гидросистемы рулевого управления (ГРУ) и комбинированной системы, где один насос питает обе эти подсистемы параллельно.

Для отдельных систем (ГНС и ГРУ):

- Программный комплекс позволяет успешно рассчитывать ключевые параметры, такие как подача и давление насоса, давления в полостях гидроцилиндров, скорости штоков, развиваемые мощности и общие КПД систем.

- Расчет теплового баланса для отдельных систем выявил, что система ГНС (Пример 1) при заданных параметрах склонна к значительному перегреву, указывая на необходимость применения дополнительных средств охлаждения или пересмотра параметров бака. Система ГРУ (Пример 2), являясь менее мощной, показала удовлетворительный тепловой режим при использовании общего гидробака.

- Полученные значения КПД для отдельных систем (около 70-74%) являются типичными для объемных гидроприводов и свидетельствуют о корректности заложенных методик расчета потерь.

Для комбинированной системы (Пример 3):

- Расчет показал, что при параллельном питании двух потребителей с различными требуемыми рабочими давлениями от одного нерегулируемого насоса через простой разветвитель потока, давление на выходе насоса определяется наиболее нагруженной веткой.

- Это приводит к значительным потерям энергии на дросселирование в менее нагруженной ветке, что выражается в очень низком общем КПД системы (около 47%) и чрезвычайно высоком тепловыделении. Расчетная установившаяся температура рабочей жидкости в этом случае достигла недопустимо высоких значений, однозначно указывая на неэффективность и потенциальную неработоспособность такой конфигурации без кардинальных изменений или мощной системы охлаждения.

- Данный пример наглядно демонстрирует важность правильного подбора компонентов и схемных решений при проектировании многопоточных гидросистем.

Программа корректно обрабатывает данные о модулях и их соединениях, производит трассировку путей потока, включая разветвления и слияния.

Расчеты потерь давления в трубопроводах и гидроаппаратуре, а также определение рабочих параметров гидродвигателей выполняются в соответствии с заложенными теоретическими методиками.

Детализированный вывод результатов позволяет инженеру не только получить итоговые характеристики, но и проанализировать работу отдельных участков и компонентов гидросистемы, выявить «узкие места» и потенциальные проблемы.

Таким образом, представленные примеры подтверждают работоспособность программного комплекса и его пригодность для выполнения проектных расчетов объемных гидроприводов различной конфигурации. Полученные результаты подчеркивают важность комплексного подхода к проектированию.

Заключение

В ходе выполнения данного проекта были решены поставленные задачи по углублению теоретических знаний в области объемных гидроприводов и разработке программного инструмента для их проектирования и расчета.

Основные результаты, полученные в ходе выполнения курсового проекта, заключаются в следующем:

- Систематизированы теоретические основы расчета объемных гидроприводов, включая методики определения мощности, выбора насосов, гидродвигателей и гидроаппаратуры, расчета гидрوليний, потерь давления, КПД и теплового баланса. Эта теоретическая база легла в основу алгоритмов, реализованных в программном комплексе.

- Разработан программный комплекс с клиент-серверной архитектурой, предоставляющий пользователю интерактивный графический интерфейс для построения гидравлических схем и задания параметров их элементов. Клиентская часть, созданная с использованием React и React DnD, обеспечивает удобное визуальное конструирование. Серверная часть на Node.js с Express и Mongoose реализует сложную логику анализа топологии схем, выполнения гидравлических и тепловых расчетов, а также управление хранением схем в базе данных MongoDB.

- Программный комплекс успешно автоматизирует расчеты таких ключевых параметров, как скорости потоков, числа Рейнольдса, коэффициенты трения, потери давления на трение и в местных сопротивлениях, рабочие давления в полостях гидродвигателей, развиваемые ими усилия и скорости, потребляемые и полезные мощности, а также различные виды КПД. Кроме того, реализован расчет теплового баланса системы с определением установившейся температуры рабочей жидкости.

- На примерах расчета гидронавесной системы, системы рулевого управления и комбинированной гидросистемы была продемонстрирована работоспособность программного комплекса и его способность анализировать схемы различной сложности. Результаты расчетов позволяют оценить адекватность выбранных компонентов, выявить потенциальные проблемы (например, перегрев или низкий КПД) и принять обоснованные проектные решения.

- Полученные в ходе тестовых расчетов данные (например, значительное тепловыделение в комбинированной схеме при параллельном питании разнонагруженных потребителей от одного насоса) подчеркивают важность тщательного анализа и выбора оптимальных схемных решений для обеспечения высокой эффективности и надежности гидропривода.

Оценка эффективности разработанного программного комплекса показывает, что он является полезным инструментом, способным значительно сократить время и трудоемкость проектных расчетов по сравнению с ручными вычислениями. Визуальный интерфейс и автоматизация расчетов позволяют инженеру сосредоточиться на концептуальных аспектах проектирования и анализе различных вариантов.

Список используемых источников

1. Башта, Т.М. Машиностроительная гидравлика : Справочное пособие / Т.М. Башта, С.С. Руднев, Б.Б. Некрасов [и др.]. – М.: Машиностроение, 1982. – 423 с. (Этот источник указан в списке литературы к документу Проектирование гидроприводов строительных и дорожных машин.pdf и является классическим справочником).
2. Вавилов, А.В. Проектирование гидроприводов строительных и дорожных машин: Учебно-методическое пособие для студентов специальности 1-36 11 01 «Подъемно-транспортные, строительные, дорожные машины и оборудование» / А.В. Вавилов, А.Н. Смоляк. – Минск: БНТУ, 2012. – 74 с.
3. ГИДРАВЛИКА, ГИДРОМАШИНЫ И ГИДРОПРИВОДЫ: Пособие для студентов специальностей 1-36 01 05 «Машины и технология обработки металлов давлением», 1-36 01 06 «Оборудование и технология сварочного производства» / Сост.: И.В. Качанов, В.А. Ключников, И.М. Шаталов, М.К. Щербакова. – Минск: БНТУ, 2019. – 30 с.
4. ГИДРАВЛИКА, ГИДРОМАШИНЫ И ГИДРОПРИВОД: Методические указания к практическим занятиям для студентов специальности I-36 11 01 «Подъемно-транспортные, строительные, дорожные машины и оборудование» / Сост.: И.В. Лесковец, А.П. Смоляр, В.В. Кутузов. – Могилев: ГУ ВПО «Белорусско-Российский университет», 2011. – 23 с.
5. Гидравлический расчёт объёмного гидропривода с возвратно-поступательным движением выходного звена: Методические указания по выполнению расчётно-графической работы / Сост. Н.П. Жуков. – Тамбов: Изд-во ГОУ ВПО ТГТУ, 2010. – 32 с.
6. Петренко, С.М. Проектный расчет объемного гидропривода вращательного движения: Методическое пособие для студентов специальности 1-36 10 01 «Горные машины и оборудование» заочной формы обучения / С.М. Петренко. – Минск: БНТУ, 2011. – 48 с.
7. Расчет объемного гидропривода (PDF-документ [raschet_gidroprivoda.pdf](#), 14 стр., без полных выходных данных, но использовался как основа для теоретической части).
8. Требования к проекту (Word-документ [TREBOVANIA_K_PROEKTU.docx](#), описывающий общие входные и выходные параметры для программного комплекса).
9. ГОСТ 2.704–76 ЕСКД. Правила выполнения гидравлических и пневматических схем. (Упоминается в [2, стр. 8])
10. ГОСТ 12445–80 Гидроприводы объемные, пневмоприводы и смазочные системы. Давления номинальные. (Упоминается в [2, стр. 8])
11. ГОСТ 13825–80 Гидроприводы объемные и пневмоприводы. Расходы номинальные. (Упоминается в [2, стр. 9])
12. ГОСТ 12447–80 Гидроцилиндры и пневмоцилиндры. Диаметры. (Упоминается в [2, стр. 23])
13. Васильченко, В.А. Гидравлическое оборудование мобильных машин: справочник / В.А. Васильченко. – М.: Машиностроение, 1983. – 301 с. (Часто цитируемый источник по гидрооборудованию).

14. Юшкин, В.В. Основы расчета объемного гидропривода / В.В. Юшкин. – М.: Высшая школа, 1982. – 93 с. (Указан в списке литературы к документу [2]).

15. Свешников, В.К. Станочные гидроприводы: справочник / В.К. Свешников, А.А. Усов. – М.: Машиностроение, 1982. – 464 с. (Классический справочник по гидроприводам).